TropicalGT: Tropical Attention, Polyhedral Reasoning, and Bits-per-byte-Safe Training for Graph-Token Reasoning Agents

Amelie Schreiber

June 2026

Abstract

Tropical Attention, introduced at NeurIPS 2025 by Hashemi, Pasque, Teska, and Yoshida, replaces softmax-normalized dot-product attention with a max-plus, tropical-projective attention mechanism designed for neural algorithmic reasoning. The central insight is that many dynamic-programming and combinatorial-optimization value functions are tropical polynomials: maxima of affine functions, or compositions of max and ordinary addition. This paper develops *TropicalGT*, a graph-token and reasoning-agent architecture that turns that first iteration into a broader training theory. TropicalGT uses tropical attention heads as hard, auditable routing primitives; graph-token transformers as equivariant carriers of typed reasoning states; blockwise ring evaluation as a long-context execution schedule; tropical Newton polytopes as diagnostics of active proof alternatives; and bits-per-byte-gated objectives as the deployment contract. The goal is not merely to replace softmax. The goal is to make hidden reasoning trajectories behave like stable polyhedral algorithms whose active faces, margins, certificates, and compression utility can be inspected.

We give a rigorous mathematical specification of TropicalGT, including definitions of tropicalized token spaces, multi-head tropical graph attention, Hilbert-metric scores, active-face certificates, blockwise tropical scans, and structured reasoning packets. We prove equivariance, 1-Lipschitz stability, active-face preservation under quantization, exactness of blockwise max-plus evaluation, simulation theorems for dynamic-programming recurrences, and a conditional-information theorem explaining when tropical certificates can improve held-out bits-per-byte. We then propose training losses, metrics, regularizers, and curricula for algorithmic tasks, graph reasoning, retrieval, long-context reasoning, memory compression, and behavior control. The paper includes implementation-level pseudocode, PyTorch-style kernel sketches, dataset and ablation protocols, and reproducibility details. It is written as a research-program paper: the formal theorems hold under explicit finite or compact assumptions, while empirical claims are framed as hypotheses to be validated by matched ablations and score-first bits-per-byte evaluation.

Contents

1 Motivation and relation to the ToricGT program

ToricGT began with graph-token reasoning, tropical ring attention, noncommutative-torus phase channels, finite toric or BGG certificates, and a strict compression contract: auxiliary structure is useful only when it improves held-out bits-per-byte, reasoning quality, retrieval quality, behavior control, or another predeclared metric after artifact-byte accounting. The original ToricGT formulation already treated max-plus heads as dynamic-programming primitives, and later iterations developed full-context reasoning, filtered persistence complexes, sheaves, vector bundles, derived objects, MCP memory, graph-structured vector retrieval, and oracle trajectory classifiers. TropicalGT isolates and expands the tropical component of that program.

The NeurIPS 2025 Tropical Attention paper supplies the catalyst. Its central claim is that softmax attention is geometrically misaligned with combinatorial algorithms: softmax produces smooth mixtures, while dynamic programs often require decisive max/min selections over polyhedral value functions. Tropical Attention addresses this by moving attention into tropical projective space, scoring query-key agreement by a tropical Hilbert metric, aggregating by max-plus matrix-vector products, and returning the result to Euclidean coordinates for ordinary transformer processing [1]. The authors prove that multi-head Tropical Attention can approximate tropical circuits and simulate max-plus dynamic programs; the official OpenReview record lists it as a NeurIPS 2025 poster and states that MHTA stacks approximate tropical circuits and realize tropical transitive closure with polynomial resource bounds [2]. The public code repository exposes a standalone `TropicalAttention.py` kernel, experimental scripts, and eleven combinatorial task generators [3].

TropicalGT also builds on the tropical expressivity program of Su and Liu, where zero-temperature self-attention is analyzed as a tropical rational map whose query-space cells are power diagrams and whose multi-head complexity is governed by Minkowski sums of Newton polytopes [5]. The TokenGT expressivity extension sharpens this picture for graph data: graph tokenization changes the effective token count, makes vertex-edge incidence directly visible to attention, supports graph-to-graph universality on bounded domains, and clarifies the Boolean-circuit and toric-boundary edge cases [6]. TropicalGT-I turns those expressivity claims into a training and audit contract: the code logs supports, margins, structural certificates, GFlowNet paths, GraphCG directions, and filtered simplicial objects rather than treating the tropical skeleton as an invisible proof device.

TropicalGT asks the next question: what happens if tropical attention is not a local replacement for softmax, but the organizing principle for an auditable reasoning-agent stack? The answer is a model class with five layers:

1. **Tropical kernels.** Selected attention heads operate over max-plus or min-plus semirings, exposing active supports, top-two margins, and polyhedral cells.

2. **Graph-token carriers.** Nodes, edges, memory records, proof obligations, tool calls, and retrieved evidence become typed graph tokens processed equivariantly.

3. **Polyhedral diagnostics.** The active affine candidates of a tropical head form Newton polytopes and normal-fan cells; reasoning decisions become active faces.

4. **Certificate-aware training.** Dynamic-programming recurrences, shortest-path trees, subset-sum tables, proof graphs, persistence summaries, and behavior labels are converted into finite certificates and losses.

5. **Bits-per-byte deployment.** Tropical probes may remain teacher-side. They enter the deployable student only when quantized export improves held-out bits-per-byte or gives a predeclared reasoning/control gain without statistically meaningful compression regression.

The paper is also a response to a practical difficulty in reasoning-agent training. Many models can output plausible intermediate reasoning, but hidden trajectories are rarely audited. Tropical

structure gives an intermediate language between symbolic proof and opaque activation. A tropical head computes a maximum; the maximizing index is a discrete certificate. If the top-two margin is large, the certificate is stable. If the same certificate recurs across graph isomorphisms or value shifts, it is evidence of algorithmic structure. If the certificate improves future-byte prediction or verifier accuracy under strict prefix dependence, it has compression value.

Contributions. This paper contributes:

1. a self-contained formalization of TropicalGT as a graph-token transformer with tropical, softmax, hybrid, and ring-evaluated attention heads;
2. an exact mathematical dictionary from tropical attention to active faces, normal fans, dynamic-programming recurrences, and reasoning certificates;
3. theorems for equivariance, stability, active-face preservation, tropical circuit simulation, graph recurrence simulation, and bits-per-byte utility of prefix-visible certificates;
4. a practical catalogue of losses, metrics, objectives, and regularizers for training TropicalGT on algorithmic reasoning, graph reasoning, long-context retrieval, and behavior-controlled reasoning;
5. implementation details, pseudocode, reproducibility protocols, and ablation ladders designed to separate real compression/reasoning gain from ornamental mathematical structure.

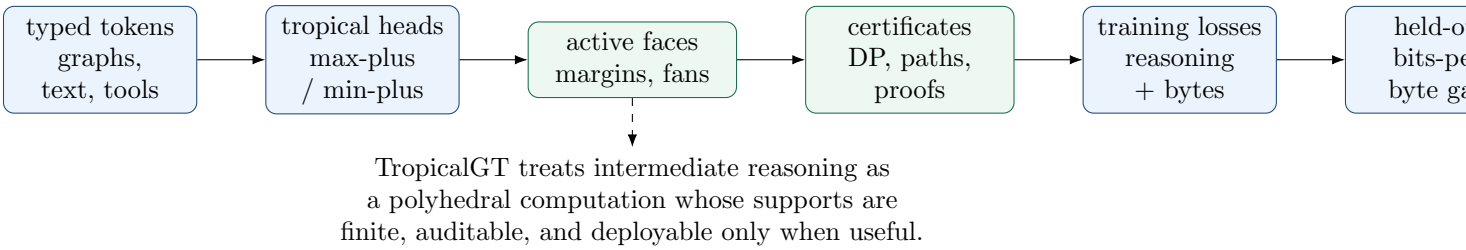

Figure 1: TropicalGT pipeline. Tropical heads expose active maximizers and margins; these become finite certificates and training losses; deployment is controlled by held-out bits-per-byte and reasoning metrics.

2 Review of Tropical Attention and its limitations

2.1 The first iteration: tropical attention at NeurIPS 2025

The Tropical Attention paper is motivated by neural algorithmic reasoning. Dynamic programming algorithms for combinatorial optimization frequently combine max/min with ordinary addition. In the max-plus semiring, a recurrence such as

$$D_{t+1}(v) = \max_{u \to v} \{D_t(u) + w(u, v)\} \tag{1}$$

is linear: addition in the semiring is maximum, multiplication is ordinary addition. The value function is a tropical polynomial, i.e. a maximum of affine monomials. This matches the polyhedral geometry of optimal alternatives. The paper contrasts this with softmax dot-product attention, where every key receives positive mass and sharp choices are smoothed. The arXiv version states that Tropical Attention operates natively in the max-plus semiring and is intended to preserve sharp, scale-invariant reasoning that softmax blurs [1].

The mechanism has four conceptual steps:

1. map Euclidean token embeddings to tropical projective coordinates by a valuation-like map;
2. form tropical linear projections for queries, keys, and values;
3. compute query-key agreement using the tropical Hilbert projective metric;
4. aggregate values by a tropical matrix-vector product and devaluate back to Euclidean coordinates.

The paper's public slides summarize the key attention equation as a projective selection: the tropical context picks the value that best aligns projectively with the query using a Hilbert distance and a max-plus reduction [4]. It also emphasizes three requirements for algorithmic alignment: hard argmax/argmin operations, piecewise-linear decision boundaries, and robustness to perturbations. Earlier tropical decision-boundary work likewise showed how ReLU network boundaries sit inside tropical hypersurfaces controlled by Newton-polytope data [12]. Those are exactly the properties TropicalGT makes explicit and auditable.

2.2 What TropicalGT changes

The first iteration proves that tropical attention can simulate tropical circuits and performs well on controlled algorithmic tasks. TropicalGT adds five extensions.

First, it treats tropical attention as one head family inside a graph-token model. Some heads should remain softmax because natural-language and diffuse semantic retrieval sometimes require mixture. TropicalGT uses a typed head bank: tropical heads for algorithmic max/min, softmax heads for distributional semantics, and hybrid heads when the model must learn a soft-to-hard curriculum.

Second, it makes the active maximizer a first-class supervision object. The Tropical Attention paper visualizes sharp attention maps for Quickselect and Knapsack; TropicalGT trains on the support itself when a certificate is available. The support is not merely an explanation. It is a discrete variable whose entropy, stability, and agreement with a teacher can be measured.

Third, it uses tropical geometry beyond max-plus arithmetic. A head with affine candidates $a_r \cdot h + b_r$ has a lifted Newton polytope $\mathrm{Conv}(a_r, b_r)$. The hidden state h exposes a face of this polytope. The corresponding normal fan partitions hidden space into decision cells. The full TropicalGT geometric program records active faces, wall distances, and bends when fan geometry is available; the v1 implementation logs active supports, top-two margins, support entropies, and margin-threshold wall-hit proxies as certificate diagnostics.

Fourth, it connects tropical attention to reasoning trajectories and long context. A tropical head can be evaluated blockwise because maximum is associative and commutative. This gives exact support scans over large context shards. For long-context reasoning, the support of a tropical query over evidence blocks is a compact proof-of-use certificate.

Fifth, it retains the ToricGT compression contract. Every tropical loss is staged. Audit-only probes come first; loss-enabled probes require exact/surrogate agreement; deployed probes must improve held-out bits-per-byte or predeclared reasoning/control metrics after artifact accounting.

2.3 Limitations inherited from the source paper

The Tropical Attention paper itself notes that its experiments were on combinatorial algorithms and had not yet demonstrated scale to generative autoregressive language tasks; it also flags computational and memory overhead from tropical operations and the Hilbert metric [1]. TropicalGT treats those as design constraints:

- use tropical heads selectively, not universally;
- implement max-plus kernels with blockwise scans and sparsity masks;
- train on algorithmic and graph-structured slices before mixing into natural text;

- distill tropical support codes into compact students;
- report bits-per-byte and artifact bytes, not only task accuracy.

3 Mathematical foundations

3.1 The tropical semiring and tropical polynomials

Definition 3.1 (Max-plus semiring)**.** *The max-plus tropical semiring is*

$$\mathbb{T}_{\max} = \mathbb{R} \cup \{-\infty\}, \qquad a \oplus_{\text{trop}} b = \max(a, b), \qquad a \odot_{\text{trop}} b = a + b. \tag{2}$$

The additive identity is $-\infty$ and the multiplicative identity is 0. Matrix multiplication over $\mathbb{T}_{\max}$ is

$$(A \odot_{\text{trop}} B)_{ij} = \max_k (A_{ik} + B_{kj}). \tag{3}$$

The min-plus semiring is obtained by replacing maximum with minimum and using + as multiplication.

Definition 3.2 (Tropical polynomial)**.** *A tropical polynomial in $x \in \mathbb{R}^d$ is a function*

$$f(x) = \bigoplus_{r=1}^{R} c_r \odot_{\text{trop}} x_1^{\odot_{\text{trop}} \alpha_{r1}} \odot_{\text{trop}} \cdots \odot_{\text{trop}} x_d^{\odot_{\text{trop}} \alpha_{rd}} = \max_{1 \le r \le R} \{c_r + \langle \alpha_r, x \rangle\}, \tag{4}$$

where $c_r \in \mathbb{R}$ and $\alpha_r \in \mathbb{N}^d$. Thus tropical polynomials are convex piecewise-linear functions. The active set at x is

$$A_f(x) = \arg\max_r \{c_r + \langle \alpha_r, x \rangle\}. \tag{5}$$

Lemma 3.3 (Tropical polynomials are 1-Lipschitz with respect to coefficient perturbation)**.** *Let $f(x) = \max_r (c_r + \langle \alpha_r, x \rangle)$ and $g(x) = \max_r (d_r + \langle \alpha_r, x \rangle)$. If $\max_r |c_r - d_r| \le \varepsilon$, then $|f(x) - g(x)| \le \varepsilon$ for all x.*

Proof. For each r, $c_r + \langle \alpha_r, x \rangle \le d_r + \langle \alpha_r, x \rangle + \varepsilon$. Taking maxima gives $f(x) \le g(x) + \varepsilon$. Reversing the roles of c and d gives $g(x) \le f(x) + \varepsilon$. The result follows. □

3.2 Tropical projective space and Hilbert metric

Tropical attention uses projective invariance: adding a constant to every coordinate of a vector should not change its meaning. Let $\mathbf{1} = (1, \ldots, 1)$. Two vectors $x, y \in \mathbb{R}^d$ are tropically projectively equivalent if $x - y = \lambda \mathbf{1}$ for some scalar λ. The quotient $\mathbb{R}^d / \mathbb{R}\mathbf{1}$ is tropical projective space. A convenient section is

$$\pi(x) = x - \max_i x_i \, \mathbf{1}, \qquad \max_i \pi(x)_i = 0. \tag{6}$$

The tropical Hilbert projective metric is

$$d_H(x, y) = \max_i (x_i - y_i) - \min_i (x_i - y_i). \tag{7}$$

It is invariant under adding constants to either argument.

Lemma 3.4 (Projective invariance)**.** *For any $x, y \in \mathbb{R}^d$ and $\alpha, \beta \in \mathbb{R}$,*

$$d_H(x + \alpha \mathbf{1}, y + \beta \mathbf{1}) = d_H(x, y). \tag{8}$$

Proof. The coordinate differences become $(x_i - y_i) + (\alpha - \beta)$. Both the maximum and minimum are shifted by $\alpha - \beta$, so their difference is unchanged. □

Lemma 3.5 (Sup-norm control of Hilbert distance)**.** *If $\|x - x'\|_\infty \le \varepsilon$ and $\|y - y'\|_\infty \le \varepsilon$, then*

$$|d_H(x, y) - d_H(x', y')| \le 4\varepsilon. \tag{9}$$

Proof. Let $z = x - y$ and $z' = x' - y'$. Then $\|z - z'\|_\infty \le 2\varepsilon$. The maximum of coordinates changes by at most 2ε, and the minimum changes by at most 2ε. The range changes by at most 4ε. □

3.3 Tropical attention as projective nearest-neighbor selection

Let $X \in \mathbb{R}^{L\times d}$ be token states. A tropicalization map $\nu_\lambda : \mathbb{R}^d \to \mathbb{R}^d/\mathbb{R}\mathbf{1}$ may be implemented by

$$\nu_\lambda(x) = \pi\big(\log(\mathrm{softplus}(A_\lambda x + b_\lambda) + \varepsilon_0)\big), \tag{10}$$

where A_λ, b_λ are learned and $\varepsilon_0 > 0$ prevents $\log 0$. For a tropical head, define projected queries, keys, and values by ordinary affine maps followed by projection, or by tropical linear maps. TropicalGT permits both; the simplest trainable head is

$$\begin{aligned} Q_i &= \pi(X_i W_Q), \qquad K_j = \pi(X_j W_K), \qquad V_j = X_j W_V, \\ S_{ij} &= -d_H(Q_i, K_j) + B_{ij}, \\ Y_{ic} &= \max_{j\in\mathcal{N}(i)} \{S_{ij} + V_{jc}\}, \end{aligned} \tag{11}$$

where B_{ij} is a structural bias and $\mathcal{N}(i)$ is a mask-defined key set.

Definition 3.6 (Active support and margin)**.** *The active support of token i and output channel c is*

$$A_{ic} = \arg\max_{j\in\mathcal{N}(i)} \{S_{ij} + V_{jc}\}. \tag{12}$$

If the maximizer is unique, the top-two margin is

$$\gamma_{ic} = z_{ic}^{(1)} - z_{ic}^{(2)}, \qquad z_{icj} = S_{ij} + V_{jc}, \tag{13}$$

where $z_{ic}^{(1)}$ and $z_{ic}^{(2)}$ are the largest and second-largest candidates.

Theorem 3.7 (Active support stability)**.** *Assume $\|S - S'\|_\infty \le \varepsilon_S$ and $\|V - V'\|_\infty \le \varepsilon_V$. If (i, c) has a unique maximizer under (S, V) with margin $\gamma_{ic} > 2(\varepsilon_S + \varepsilon_V)$, then the unique maximizer is unchanged under (S', V'). Also $\|Y - Y'\|_\infty \le \varepsilon_S + \varepsilon_V$.*

Proof. For every candidate j, the value $S_{ij} + V_{jc}$ changes by at most $\varepsilon_S + \varepsilon_V$. The maximum of a finite set is 1-Lipschitz in sup norm, giving the output bound. If $j^\star$ is the unique maximizer and $j \ne j^\star$, then after perturbation

$$z'_{icj^\star} - z'_{icj} \ge z_{icj^\star} - z_{icj} - 2(\varepsilon_S + \varepsilon_V) \ge \gamma_{ic} - 2(\varepsilon_S + \varepsilon_V) > 0. \tag{14}$$

Thus $j^\star$ remains the unique maximizer. □

3.4 Newton polytopes and active faces

When a tropical head is written as $\psi(h) = \max_r\{\langle a_r, h\rangle + b_r\}$, its candidate slopes form a lifted Newton polytope

$$P_\psi = \mathrm{Conv}\{(a_r, b_r) : 1 \le r \le R\} \subset \mathbb{R}^{d+1}. \tag{15}$$

The hidden state $(h, 1)$ exposes a face of this polytope. This gives a geometric certificate for each decision.

Proposition 3.8 (Active sets are normal-fan cells)**.** *For a fixed active subset $A \subseteq \{1, \dots, R\}$, the region*

$$C_A = \{h : \langle a_r - a_s, h\rangle + b_r - b_s \ge 0 \text{ for all } r \in A,\ s \notin A\} \tag{16}$$

is a polyhedron. If $A = A_\psi(h)$, then A is the vertex set of the exposed face of P_ψ maximizing the linear functional $(a, b) \mapsto \langle a, h\rangle + b$. The cells C_A refine the normal fan of P_ψ.

Proof. The inequalities are linear, hence their intersection is a polyhedron. A point (a_r, b_r) lies on the exposed face for functional $(h, 1)$ exactly when it attains the maximum value of $\langle a, h\rangle + b$. Thus the active set is the exposed face. As h varies, exposed faces of a polytope are organized by its normal fan. □

4 TropicalGT architecture

4.1 Typed graph-token domains

A reasoning problem is not always a string. It may be a graph of facts, proof obligations, memory records, tool calls, retrieved documents, or intermediate states. TropicalGT uses graph-tokenization to preserve this structure.

Definition 4.1 (Typed attributed reasoning graph)**.** *A typed attributed reasoning graph is*

$$G = (V, E, s, t, \tau_V, \tau_E, x, e, m_V, m_E), \tag{17}$$

where V is a finite set of vertex tokens, E is a finite set of edge tokens, $s, t : E \to V$ are endpoint maps, τ_V, τ_E are finite type maps, $x_v \in \mathbb{R}^{d_v}$ and $e_a \in \mathbb{R}^{d_e}$ are attributes, and m_V, m_E are validity masks for padding.

Types include problem tokens, numbers, graph nodes, graph edges, proof claims, proof dependencies, retrieved passages, tool calls, verifier messages, memory summaries, and behavior-control states. The token matrix is

$$Z = T(G) = [Z_V; Z_E] \in \mathbb{R}^{(n+m)\times d}. \tag{18}$$

A node token has the form

$$Z_v = \phi_V(x_v, \tau_V(v)) + \eta_v + \kappa_v + \theta_v, \tag{19}$$

and an edge token has the form

$$Z_a = \phi_E(e_a, \tau_E(a)) + \psi(\eta_{s(a)}, \eta_{t(a)}) + \zeta_a + \kappa_a + \theta_a. \tag{20}$$

Here η and ζ are equivariant identifiers, κ are structural features, and θ are optional tropical or toric chart features.

Theorem 4.2 (Graph-token equivariance of TropicalGT blocks)**.** *Assume token identifiers, masks, and structural features transform equivariantly under graph relabeling, and assume all attention projections, tropical projections, feed-forward maps, and decoders are shared across token rows. Then a TropicalGT encoder built from softmax heads, tropical heads, hybrid heads, residuals, layer normalization, and shared decoders is graph-to-graph equivariant.*

Proof. A relabeling acts by a permutation matrix P on token rows. Shared linear maps commute with row permutations, so $Q(PZ) = PQ(Z)$, and similarly for keys and values. The attention mask transforms as $M(PZ) = PM(Z)P^{\mathsf{T}}$. Softmax attention is equivariant because row-wise softmax of PAP^{T} is $P\,\mathrm{softmax}(A)P^{\mathsf{T}}$. Tropical attention is equivariant because

$$\max_j\{(PAP^{\mathsf{T}})_{ij} + (PV)_{jc}\} = \max_{j'}\{A_{p^{-1}i,j'} + V_{j'c}\}, \tag{21}$$

which is the permuted output. Residuals, layer normalization, and shared feed-forward layers are row-wise and therefore commute with P. A composition of equivariant maps is equivariant. Shared decoders preserve equivariance; symmetric pooling gives graph-level invariance. □

4.2 Head types

TropicalGT uses a typed head bank rather than a monolithic replacement of softmax.

Softmax head.

$$Y_i^{\mathrm{soft}} = \sum_j \mathrm{softmax}_j\left(\frac{\langle Q_i, K_j\rangle}{\sqrt{d}} + M_{ij}\right) V_j. \tag{22}$$

This is useful for diffuse semantic aggregation, distributional retrieval, and uncertainty representation.

Tropical head.

$$Y_{ic}^{\mathrm{trop}} = \max_j\{-d_H(Q_i, K_j) + M_{ij} + V_{jc}\}. \tag{23}$$

This is useful for dynamic programming, hard evidence maxima, path selection, and exact support certificates.

Min-plus head.

$$Y_{ic}^{\min} = \min_j\{d_H(Q_i, K_j) - M_{ij} + V_{jc}\}. \tag{24}$$

This is useful for shortest paths, minimal costs, proof-distance penalties, and nearest valid certificate retrieval.

Hybrid head. A hybrid head computes both soft and tropical contexts and gates them:

$$Y_i = g_i \odot Y_i^{\mathrm{trop}} + (1 - g_i) \odot Y_i^{\mathrm{soft}}, \qquad g_i = \sigma(w^{\mathsf{T}} h_i + b). \tag{25}$$

The gate is supervised when task metadata says a step is algorithmic, retrieval-heavy, expository, or uncertain.

Ring tropical head. For long contexts, keys are partitioned into blocks $B_1, \ldots, B_R$. The head computes block summaries

$$Y_{ic}^{(r)} = \max_{j \in B_r}\{S_{ij} + V_{jc}\}, \qquad Y_{ic} = \max_r Y_{ic}^{(r)}. \tag{26}$$

This is exact in real arithmetic and supports streaming.

Theorem 4.3 (Exact blockwise tropical ring evaluation)**.** *Let $\{B_r\}_{r=1}^R$ be a partition of the key indices. Then*

$$\max_j\{S_{ij} + V_{jc}\} = \max_r \max_{j \in B_r}\{S_{ij} + V_{jc}\}. \tag{27}$$

Moreover, if each block returns its local argmax set, the union of local argmax sets among globally maximal blocks is exactly the global argmax set.

Proof. The first identity is associativity and commutativity of finite maximum over a disjoint union. A key j is globally active exactly when its block maximum equals the global maximum and j is locally active inside that block. □

4.3 TropicalGT layer

A TropicalGT layer has the form

$$H^{\ell+1/2} = H^{\ell} + \sum_{a \in \mathcal{A}_\ell} W_a \, \mathrm{Head}_a(H^{\ell}, M), \tag{28}$$

$$H^{\ell+1} = \mathrm{LN}\left(H^{\ell+1/2} + \mathrm{FFN}_\ell(H^{\ell+1/2})\right), \tag{29}$$

where $\mathcal{A}_\ell$ includes softmax, tropical, min-plus, and hybrid heads. Low-rank probe heads may output active support ids, top-two margins, path predecessor labels, or fan-cell labels.

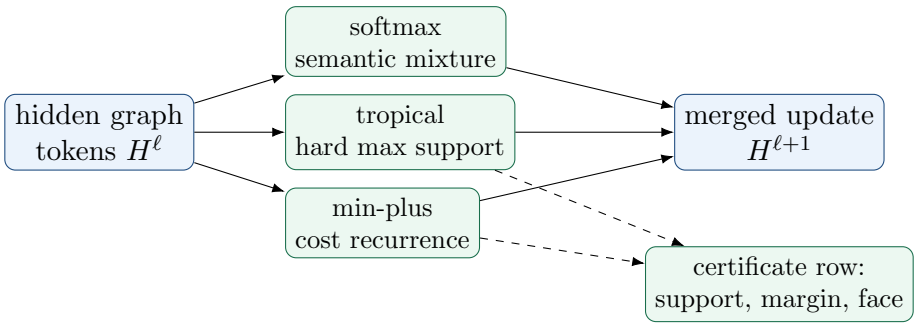

Figure 2: One TropicalGT layer. Tropical and min-plus heads supply hard supports and dynamic-programming inductive bias; softmax heads retain diffuse semantic modeling.

5 Reasoning trajectories as tropical dynamical objects

5.1 Reasoning packets

A reasoning agent fills a trajectory. In a graph-token setting, a step is not a sentence but a packet.

Definition 5.1 (Tropical reasoning packet)**.** *A tropical reasoning packet at time t is*

$$R_t = (G_t, H_t, U_t, E_t, C_t, A_t, \Gamma_t, B_t), \tag{30}$$

where G_t is a typed graph, H_t hidden tokens, U_t retrieved or tool-derived evidence, E_t verifier fields, C_t behavior-control coordinates, A_t active tropical supports, Γ_t fan-cell and margin summaries, and B_t output bytes or byte logits. A TropicalGT model induces a stochastic kernel

$$P_\theta(R_{t+1} \mid R_{\le t}, x) \tag{31}$$

or a deterministic update $R_{t+1} = F_\theta(R_t, x)$.

The active supports A_t are essential. They say which predecessor, evidence passage, key item, proof clause, or memory block was decisive. They let the system distinguish a plausible answer from an answer whose internal routing matches a known algorithmic certificate.

5.2 Tropical trajectory complexes

From a packet we build a filtered complex. Let V_t be tokens and let $w_t(v)$ be an importance score derived from tropical margins, verifier confidence, evidence scores, or path costs. For thresholds $\alpha_0 \le \cdots \le \alpha_m$, define

$$K_t(\alpha_p) = \{\sigma \subseteq V_t : \max_{v \in \sigma} w_t(v) \le \alpha_p \text{ and } \sigma \text{ is graph-admissible}\}. \tag{32}$$

This produces $K_t(\alpha_0) \subseteq \cdots \subseteq K_t(\alpha_m)$. For a path-finding task, low thresholds may include source-connected nodes with low tentative distance; for retrieval, they may include evidence nodes with high tropical support and valid provenance; for behavior control, they may include tone/personality states within a corridor.

Proposition 5.2 (Support-stable filtrations)**.** *Suppose the vertex scores w_t are continuous functions of hidden states and tropical candidate values. If all active supports used in w_t have margins at least $\gamma > 0$, then any perturbation of hidden states that changes every candidate value by at most $\gamma/4$ leaves all active supports unchanged and changes each score by at most a fixed Lipschitz multiple of the perturbation. Therefore threshold grids with spacing exceeding twice this score perturbation have identical filtration membership.*

Proof. Active supports are unchanged by the active support stability theorem. Conditional on the same support, each score is an affine or smooth local function of the hidden state, hence Lipschitz on the bounded evaluation domain. If each score changes by less than half the gap between adjacent thresholds, no vertex crosses a threshold, so all simplices have unchanged membership. □

5.3 Tropical hidden Markov dynamics

A useful mental model is a finite-state abstraction. Quantize each packet by its active cell and certificate:

$$q(R_t) = (\text{cell}(H_t), A_t, \Gamma_t, \text{verifier bin}, \text{behavior bin}). \tag{33}$$

If the quantization is finite, repeated reasoning induces a Markov chain on certificates. This permits ergodic diagnostics: occupation of bad behavior chambers, recurrence of low-margin cells, entropy rate of supports, and average bits-per-byte cost per step.

Theorem 5.3 (Ergodic average for finite tropical certificate chains)**.** *Let $(Q_t)_{t \ge 0}$ be an irreducible aperiodic finite Markov chain induced by quantized TropicalGT reasoning packets, with stationary distribution π. For any bounded certificate observable φ,*

$$\frac{1}{T} \sum_{t=1}^{T} \varphi(Q_t) \to \sum_{q} \pi(q)\varphi(q) \tag{34}$$

almost surely. In particular, time-averaged tropical margin, support entropy, verifier error, and bits-per-byte contribution converge to stationary expectations.

Proof. This is the standard strong law for irreducible aperiodic finite Markov chains. The assumption ensures a unique stationary distribution and positive recurrence. Applying the Markov-chain ergodic theorem to bounded φ gives the claim. □

This theorem is intentionally modest. It does not imply a trained model is good. It says that once the certificate dynamics are finite and stable, empirical time averages are meaningful audit targets.

6 Dynamic programming simulation and graph reasoning

6.1 Bellman recurrences

Let $D_t(v)$ be a max-plus value on a directed graph. The recurrence

$$D_{t+1}(v) = \max\left(D_t(v), \max_{u \to v}\{D_t(u) + w(u,v)\}\right) \tag{35}$$

can be implemented by a tropical head with query token v, key set containing v and incoming neighbors, and value $D_t(u) + w(u,v)$. For shortest paths, replace max by min or negate distances.

Theorem 6.1 (TropicalGT simulates bounded Bellman-Ford)**.** *Consider a family of directed graphs with at most n vertices and real edge weights in a bounded interval. Fix a source s. There exists a TropicalGT stack with $O(n)$ min-plus heads per layer and depth $n-1$ that exactly computes single-source shortest-path distances on graphs with no negative cycles, assuming exact real arithmetic and a mask giving incoming edges.*

Proof. The Bellman-Ford recurrence for distances is

$$d_{t+1}(v) = \min\left(d_t(v), \min_{u \to v}\{d_t(u) + w(u,v)\}\right), \qquad d_0(s) = 0, \tag{36}$$

with $d_0(v) = +\infty$ for $v \neq s$. A min-plus head for each vertex v masks keys to v and its incoming neighbors and returns the displayed minimum. All vertices update in parallel. After $n-1$ iterations, Bellman-Ford is exact in graphs without negative cycles because a shortest simple path uses at most $n-1$ edges. Thus depth $n-1$ suffices. □

6.2 Tropical transitive closure

Let $A \in \mathbb{T}^{n \times n}$ be a tropical adjacency matrix. The finite Kleene star is

$$A_T^\star = I \oplus_{\text{trop}} A \oplus_{\text{trop}} A^{\odot_{\text{trop}} 2} \oplus_{\text{trop}} \cdots \oplus_{\text{trop}} A^{\odot_{\text{trop}} T}. \tag{37}$$

It encodes best path weights of length at most T. Tropical Attention's proof appendix uses the same idea to show that MHTA can learn tropical transitive closure [1]. TropicalGT uses this result in graph-token form.

Corollary 6.2 (Graph-token tropical closure)**.** *Given a directed graph with n valid vertex tokens and a mask for legal edges, a TropicalGT stack with min-plus or max-plus heads can compute horizon-T path closure using T layers and $O(n)$ vertex-parallel heads per layer. If the task only requires reachability rather than weights, Boolean semiring values can be embedded as 0 and $-\infty$ and computed with max-plus logic.*

6.3 Quickselect as active order statistic

Quickselect asks for the k-th order statistic. A tropical head can implement a hard order-statistic certificate by comparing candidates via rank features. Let $r_i = \#\{j : x_j \leq x_i\}$. The target token is active when $|r_i - k|$ is minimal, with tie-breaking by index. A min-plus head over candidate tokens can compute

$$Y = \min_i\{|r_i - k| + \epsilon i\}. \tag{38}$$

The support identifies the selected item. The training loss should include both classification and support agreement:

$$\mathcal{L}_{\text{qs}} = \text{CE}(\hat{y}, y) + \lambda_{\text{supp}}\,\text{CE}(\hat{A}, A^\star) + \lambda_\gamma \max(0, \gamma_0 - \gamma). \tag{39}$$

The margin term is essential for length generalization: the model should not merely classify correctly at length 8; it should produce a stable support at length 1024.

6.4 Knapsack as tropical table compression

For 0/1 knapsack, the dynamic program is

$$D_{i,c} = \max(D_{i-1,c}, D_{i-1,c-w_i} + v_i). \tag{40}$$

TropicalGT can represent rows of the DP table as tokens and use a tropical head to select between skip and take alternatives. However, full tables are expensive. The model should learn compressed table certificates:

$$Z_i = \text{Compress}\{(c, D_{i,c}) : c \in [0, C]\}, \tag{41}$$

where compression may be a set of breakpoints of a piecewise-linear envelope. The certificate loss compares the envelope rather than all capacities:

$$\mathcal{L}_{\text{env}} = \sum_{c \in \mathcal{C}_{\text{probe}}} \left| \hat{D}_i(c) - D_i(c) \right| + \lambda_{\text{bp}} |\widehat{\text{breakpoints}}|. \tag{42}$$

This connects directly to bits-per-byte: a small breakpoint code that predicts future proof or answer tokens is more deployable than a full DP table.

7 Compression theory: when tropical certificates help bits-per-byte

TropicalGT uses structural losses only when they reduce future uncertainty or improve reasoning/control without compression regression. The right mathematical quantity is conditional mutual information.

Definition 7.1 (Prequential bits-per-byte)**.** *For a byte sequence $b_{1:T}$, the prequential bits-per-byte loss is*

$$\mathcal{L}_{\text{bits-per-byte}}(\theta; b_{1:T}) = -\frac{1}{T} \sum_{t=1}^{T} \log_2 p_\theta(b_t \mid b_{<t}). \tag{43}$$

The score is valid only when predictions for b_t depend on prefix-visible information.

Theorem 7.2 (Certificate side-information identity)**.** *Let B_t be the next byte, $\mathcal{F}_t$ the prefix-visible sigma-field, and Z_t a prefix-visible tropical certificate. The optimal expected log-loss improvement from observing Z_t is*

$$H(B_t \mid \mathcal{F}_t) - H(B_t \mid \mathcal{F}_t, Z_t) = I(B_t; Z_t \mid \mathcal{F}_t) \tag{44}$$

bits. If storing or computing Z_t costs c_t bits amortized per prediction, its net compression utility is positive only if

$$I(B_t; Z_t \mid \mathcal{F}_t) > c_t. \tag{45}$$

Proof. The Bayes-optimal expected log base-2 loss when predicting from $\mathcal{F}_t$ is $H(B_t \mid \mathcal{F}_t)$. When Z_t is also observed, it is $H(B_t \mid \mathcal{F}_t, Z_t)$. Their difference is the definition of conditional mutual information. A code or artifact costing c_t bits per prediction improves net compression only when the information gain exceeds the cost. □

This theorem is the central guardrail. A tropical support is not useful because it is elegant. It is useful if it lowers entropy of future bytes, selects correct reasoning moves, improves verifier accuracy, or prevents behavior drift at a cost justified by validation.

Definition 7.3 (Tropical certificate utility)**.** *For a certificate family $\mathcal{Z}$, define*

$$U(\mathcal{Z}) = \Delta\mathcal{L}^{\text{heldout}}_{\text{bits-per-byte}} + \lambda_A \Delta A + \lambda_R \Delta R + \lambda_C \Delta C - \rho \,\text{bytes}(\mathcal{Z}) - \lambda_T \,\text{time}(\mathcal{Z}), \tag{46}$$

where $\Delta\mathcal{L}^{\text{heldout}}_{\text{bits-per-byte}}$ is reduction in held-out bits-per-byte, A accuracy, R reasoning quality, and C control metrics. A certificate is deployable only if $U(\mathcal{Z}) > 0$ under matched ablations.

8 Losses, metrics, objectives, and regularizers

This section gives a catalogue of training objectives. Each item is designed to be implemented first as audit-only, then as a train-time auxiliary, and only later as deployable structure if it survives bits-per-byte accounting.

8.1 Objective 1: tropical support supervision

Given a teacher support $A^\star_{ic}$, minimize

$$\mathcal{L}_{\text{supp}} = \sum_{i,c} \text{CE}(\hat{p}_{ic}, A^\star_{ic}), \qquad \hat{p}_{ic} = \text{softmax}_j(\beta z_{icj}). \tag{47}$$

Although the deployed tropical head is hard, the surrogate uses a high-temperature softmax over candidate values for gradient flow. This teaches the model *which* evidence or predecessor is decisive. It is useful for Quickselect, path algorithms, proof-dependency selection, retrieval citation, and tool-output grounding.

8.2 Objective 2: top-two margin regularization

For a desired margin γ_0, use

$$\mathcal{L}_{\text{margin}} = \mathbb{E}_{i,c} \max(0, \gamma_0 - \gamma_{ic}). \tag{48}$$

This penalizes fragile decisions. It should be applied only when labels indicate a deterministic algorithmic step or a high-confidence evidence choice. In open-ended semantic generation, low margins may correctly represent ambiguity.

8.3 Objective 3: fan-cell consistency

For paired examples (x, x') related by graph isomorphism, value shift, or harmless reordering, require active-cell agreement:

$$\mathcal{L}_{\text{cell}} = \text{CE}(\hat{c}(x), c^\star) + \text{CE}(\hat{c}(x'), c^\star) + \lambda \mathbf{1}[\hat{c}(x) \neq \hat{c}(x')]. \tag{49}$$

This is a direct regularizer for size and value generalization. If a shortest-path proof is shifted by adding a constant to all edge costs, the projective cell should not change unless the optimum changes.

8.4 Objective 4: Hilbert-metric contrastive alignment

For query q, positive key k^+, and negatives k^-, use

$$\mathcal{L}_H = \max\left(0, m + d_H(q, k^+) - \min_{k^-} d_H(q, k^-)\right). \tag{50}$$

This aligns tropical projective geometry with retrieval or predecessor selection. It is preferable to Euclidean contrastive loss when global shifts of coordinates should be ignored.

8.5 Objective 5: Bellman residual loss

For a task with a known recurrence operator T, define

$$\mathcal{L}_{\text{Bell}} = \sum_t \|D_{t+1} - T(D_t)\|_1 . \tag{51}$$

The target D_t may be a teacher DP table, a compressed envelope, or a hidden projection. This loss teaches recurrence consistency, not just final-answer accuracy.

8.6 Objective 6: tropical closure distillation

Let a teacher compute a closure $A^\star$ such as all-pairs shortest paths or transitive reachability. A student predicts $\hat{A}$ from one or few MHTA layers:

$$\mathcal{L}_{\text{closure}} = \left\| \hat{A} - A^\star \right\|_1 + \lambda_{\text{tri}} \sum_{i,j,k} \max(0, \hat{A}_{ij} - \hat{A}_{ik} - \hat{A}_{kj}) \tag{52}$$

for min-plus distances. The triangle penalty enforces metric-like consistency.

8.7 Objective 7: active-path certificate loss

For shortest paths, a distance without a predecessor is incomplete. Let $\pi^\star(v)$ be the teacher predecessor. Use

$$\mathcal{L}_{\text{pred}} = \sum_v \text{CE}(\hat{\pi}(v), \pi^\star(v)) + \lambda \sum_v \left| \hat{d}(v) - \hat{d}(\hat{\pi}(v)) - w(\hat{\pi}(v), v) \right|. \tag{53}$$

This links the active support to the numeric value. It prevents a model from predicting correct distances via spurious routes.

8.8 Objective 8: tropical polynomial sparsity

For a head with candidates r, encourage small active dictionaries:

$$\mathcal{L}_{\text{sparse}} = \mathbb{E}_x |A(x)| + \lambda \sum_r \sqrt{\mathbb{E}_x \mathbf{1}[r \in A(x)] + \varepsilon}. \tag{54}$$

The first term encourages unique active decisions; the second encourages reuse of a compact candidate dictionary. This improves interpretability and may improve compression by limiting distinct support codes.

8.9 Objective 9: cell entropy and occupation balance

Define the empirical distribution of active cells $\hat{p}(c)$. Penalize both collapse and noise:

$$\mathcal{L}_{\text{occ}} = \lambda_{\text{low}} \max(0, H_{\min} - H(\hat{p})) + \lambda_{\text{high}} \max(0, H(\hat{p}) - H_{\max}). \tag{55}$$

A model that uses one cell for all examples has not learned algorithmic cases. A model that uses too many cells may be memorizing. The useful range is task-specific.

8.10 Objective 10: soft-to-tropical annealing

For hybrid heads, train a gate g_t with schedule

$$\mathcal{L}_{\text{anneal}} = \mathcal{L}_{\text{task}} + \lambda_g \mathbb{E} |g_t - g_t^\star| + \lambda_{\text{KL}} \, \text{KL}(p_{\text{soft}} \| p_{\text{trop,soft}}). \tag{56}$$

Early training may rely on softmax for gradients; later training shifts algorithmic steps to tropical heads. The gate target $g_t^\star$ is 1 for deterministic recurrence steps and 0 for diffuse semantic steps.

8.11 Objective 11: tropical Lipschitz robustness

For adversarial or random perturbations δ bounded by $\|\delta\| \leq \varepsilon$, use

$$\mathcal{L}_{\text{rob}} = \mathbb{E}_{x,\delta} \left\| f(x+\delta) - f(x) \right\|_1 + \lambda \mathbb{E}_{x,\delta} \mathbf{1}[A(x+\delta) \neq A(x)] \tag{57}$$

when the label should be invariant. This objective operationalizes the Hilbert-metric non-expansive intuition.

8.12 Objective 12: value-shift invariance

For tasks invariant under adding a constant to all relevant values, train

$$\mathcal{L}_{\text{shift}} = \mathbb{E}_{x,a} \left\| f(x + a\mathbf{1}) - f(x) \right\|^2 + \lambda \, \mathrm{CE}(A(x + a\mathbf{1}), A(x)). \tag{58}$$

This exploits tropical projective equivalence. It is useful for order statistics, argmax tasks, and relative-cost reasoning.

8.13 Objective 13: tropical retrieval support precision

For a retrieved context graph, let $E^\star$ be evidence nodes required by a verifier. A tropical retrieval head returns support A. Define

$$\mathrm{Prec}_{\text{trop}} = \frac{|A \cap E^\star|}{|A| + \varepsilon}, \qquad \mathrm{Rec}_{\text{trop}} = \frac{|A \cap E^\star|}{|E^\star| + \varepsilon}. \tag{59}$$

Use a differentiable surrogate based on soft supports. This makes retrieval accountable: the highest tropical evidence should be the evidence used by the answer.

8.14 Objective 14: graph-of-thought tropical flow conservation

Let reasoning steps form a DAG. Let F_{uv} be tropical flow along edge $u \to v$. Enforce

$$\mathcal{L}_{\text{flow}} = \sum_{v \notin \{s,t\}} \left| \max_{u \to v} F_{uv} - \max_{v \to w} F_{vw} \right|. \tag{60}$$

In max-plus terms this says the best incoming proof support should match the best outgoing use, except at sources and sinks. It discourages orphaned claims.

8.15 Objective 15: tropical contradiction barrier

For contradictory claim nodes a, b, their simultaneous high support is forbidden:

$$\mathcal{L}_{\text{contra}} = \sum_{(a,b) \in \mathcal{R}_{\text{contra}}} \max(0, s_a + s_b - \tau)^2. \tag{61}$$

The tropical version uses $s_a \oplus_{\text{trop}} s_b$ and active support; contradictory claims should not both be among top supports of the same proof step.

8.16 Objective 16: bits-per-byte support utility gate

For a support code Z_t, estimate conditional utility by a controlled ablation:

$$\widehat{\Delta}_{\text{bits-per-byte}}(Z) = \mathcal{L}_{\text{bits-per-byte}}(\text{student without } Z) - \mathcal{L}_{\text{bits-per-byte}}(\text{student with } Z) - \rho \, \text{bytes}(Z). \tag{62}$$

The code is allowed in deployment only if $\widehat{\Delta}_{\text{bits-per-byte}}(Z) > 0$ with confidence under matched seeds.

8.17 Objective 17: tropical proof-step verifier

For proof tasks, require each step to have an active predecessor set satisfying a verifier relation:

$$\mathcal{L}_{\text{proof}} = \sum_t \mathrm{CE}(\hat{y}_t, y_t) + \lambda \sum_t \mathbf{1}[\text{Verifier}(A_t, \text{claim}_t) = 0]. \tag{63}$$

A differentiable proxy scores whether active supports entail the current claim. The exact verifier runs on sampled batches.

8.18 Objective 18: context-block max-support stopping

Let M_t be the maximum evidence support remaining outside the current context. Stop when

$$M_t - \max_{j \in \text{context}} S_j < \delta \tag{64}$$

and verifier confidence is high. Train a stopping head with

$$\mathcal{L}_{\text{stop}} = \text{CE}(\hat{s}_t, s_t^\star) + \lambda \frac{\ell_t}{L_C}. \tag{65}$$

This prevents gratuitous long-context expenditure.

8.19 Objective 19: quantized active-face preservation

Fake-quantize tropical projections $\hat{Q}, \hat{K}, \hat{V}$. Penalize support flips:

$$\mathcal{L}_{\text{qface}} = \mathbb{E}\mathbf{1}[A(Q,K,V) \neq A(\hat{Q},\hat{K},\hat{V})] + \lambda \left\| Y - \hat{Y} \right\|_1. \tag{66}$$

The active support stability theorem gives a margin-based sufficient condition; the loss trains the model to satisfy it.

8.20 Objective 20: behavior-corridor tropical routing

Let behavior coordinates include tone, compliance, uncertainty, verbosity, and safety. For a desired corridor K, define

$$\mathcal{L}_{\text{beh}} = \text{dist}(C_t, K)^2 + \lambda \mathbf{1}[A_t \cap A_{\text{unsafe}} \neq \emptyset]. \tag{67}$$

Tropical supports identify the memory, policy, or evidence block that dominated behavior. The loss is used only with safety and evaluation audits; it should not be optimized blindly.

8.21 Objective 21: tropical Newton polytope volume control

Let $P_h = \text{Conv}\{a_r : r \in A(h)\}$ be the active-face polytope. Define

$$\mathcal{L}_{\text{vol}} = \mathbb{E}_h \max(0, \text{Vol}(P_h) - V_{\max}). \tag{68}$$

Large active faces mean many tied alternatives; this may be appropriate in ambiguous tasks but harmful in deterministic algorithms. Volume control helps enforce decisive routing.

8.22 Objective 22: tropical tableau and sorting certificate

Many order-statistic and scheduling tasks admit Young-tableau-like sorted summaries. Let $T(x)$ be a canonical tableau of ranks or intervals. Train a tropical head to recover row/column predecessor supports:

$$\mathcal{L}_{\text{tab}} = \text{CE}(\hat{T}, T^\star) + \lambda \sum_{(i,j)} \max(0, T_{i,j} - T_{i,j+1}) + \lambda \sum_{(i,j)} \max(0, T_{i,j} - T_{i+1,j}). \tag{69}$$

This supplies a combinatorial certificate for sorted reasoning. It is a bridge between tropical attention and representation-combinatorics.

8.23 Objective 23: tropical memory recurrence

For memory retrieval, let m_t be a memory state and z_t a retrieved support code. Train

$$m_{t+1} = m_t \oplus_{\text{trop}} (W \odot_{\text{trop}} z_t), \qquad \mathcal{L}_{\text{mem}} = \left\| \hat{m}_{t+1} - m^{\star}_{t+1} \right\|_1 + \lambda \, \text{bytes}(z_t). \tag{70}$$

This makes memory update a max-plus accumulation of reusable evidence rather than an unconstrained text append.

8.24 Objective 24: tropical circuit description length

Let a task be solved by a learned tropical circuit with H heads, L layers, and C distinct active support patterns. Define

$$\text{TDL} = \alpha H + \beta L + \rho \log(1 + C) + \mathcal{L}_{\text{task}}. \tag{71}$$

This is a tropical minimum-description-length objective. It encourages simple circuits that still solve the task and transfer.

9 Training paradigms

9.1 Paradigm A: drop-in MHTA replacement

The simplest paradigm is the one used in the Tropical Attention paper: replace every softmax self-attention block in a shallow transformer with MHTA while keeping depth, width, and number of heads comparable. This is the right first experiment for canonical algorithmic tasks. TropicalGT uses it as an audit baseline, not necessarily as the final architecture.

Listing 1: Minimal PyTorch-style tropical head.

```
def tropical_head(X, Wq, Wk, Wv, mask=None, eps=1e-6):
    # X: [B, L, D]
    Q = projective(torch.log(torch.nn.functional.softplus(X @ Wq) + eps))
    K = projective(torch.log(torch.nn.functional.softplus(X @ Wk) + eps))
    V = X @ Wv
    # Hilbert distance: max(Q-K)-min(Q-K) over feature dimension
    diff = Q[:, :, None, :] - K[:, None, :, :]
    dH = diff.max(dim=-1).values - diff.min(dim=-1).values
    S = -dH
    if mask is not None:
        S = S + mask
    # max-plus aggregation by channel
    Y = (S[:, :, :, None] + V[:, None, :, :]).max(dim=2).values
    support = (S[:, :, :, None] + V[:, None, :, :]).argmax(dim=2)
    return Y, support
```

9.2 Paradigm B: tropical-supervised graph-token training

When graph structure is known, use graph tokens and certificate labels. The loss is

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda_{\text{supp}} \mathcal{L}_{\text{supp}} + \lambda_{\text{margin}} \mathcal{L}_{\text{margin}} + \lambda_{\text{Bell}} \mathcal{L}_{\text{Bell}}. \tag{72}$$

This paradigm is ideal for shortest paths, reachability, SCC, proof graphs, retrieval citation, and tool-dependency graphs.

9.3 Paradigm C: teacher-side tropical circuit distillation

A large teacher runs expensive tropical probes, exact dynamic programs, or long-context ring scans. It emits compact support codes Z. A student receives either no code, shuffled code, or real code. The deployment question is whether real code improves validation after bytes:

$$\mathcal{L}_{\text{distill}} = \mathrm{KL}(p_T(\cdot \mid x) \| p_S(\cdot \mid x, Z)) + \lambda_Z \,\text{bytes}(Z) + \lambda_{\text{bits-per-byte}} \mathcal{L}_{\text{bits-per-byte}}. \tag{73}$$

9.4 Paradigm D: tropical retrieval for long context

In long-context settings, tropical attention is a retrieval and provenance primitive. Each block B_r returns a support summary:

$$(m_r, j_r, c_r) = \left(\max_{j \in B_r} z_j, \arg\max_{j \in B_r} z_j, \text{code}(B_r) \right). \tag{74}$$

The global maximum over blocks gives exact support. A context filler includes only blocks whose marginal support beats token cost.

Listing 2: Blockwise tropical retrieval scan.

```
def tropical_block_scan(query, blocks, score_fn, value_fn):
    best_score = -float("inf")
    best_value = None
    best_ref = None
    summaries = []
    for r, block in enumerate(blocks):
        scores = score_fn(query, block) # [block_len]
        idx = scores.argmax()
        val = value_fn(block[idx])
        summaries.append((scores[idx], r, idx))
        if scores[idx] > best_score:
            best_score = scores[idx]
            best_value = val
            best_ref = (r, int(idx))
    return best_value, best_ref, summaries
```

9.5 Paradigm E: bits-per-byte-gated tropical curriculum

The recommended training curriculum is:

1. train a dense baseline with no tropical auxiliaries;
2. add tropical probes audit-only and log supports/margins;
3. turn on support and margin losses for synthetic algorithmic data;
4. add graph-token and retrieval tasks;
5. train a teacher with full tropical certificates;
6. distill compact support codes into a student;
7. export quantized kernels only if bits-per-byte and reasoning gates pass.

10 Detailed examples

10.1 Worked example: a three-node shortest path

Let vertices be s, a, t and min-plus edge weights

$$w(s,a) = 2, \quad w(a,t) = 3, \quad w(s,t) = 6. \tag{75}$$

Initialize $d_0(s) = 0$, $d_0(a) = d_0(t) = +\infty$. After one step:

$$d_1(a) = \min(+\infty, 0+2) = 2, \tag{76}$$

$$d_1(t) = \min(+\infty, 0+6) = 6. \tag{77}$$

After two steps:

$$d_2(t) = \min(6, d_1(a)+3) = 5. \tag{78}$$

The active predecessor support for t changes from s to a. A TropicalGT head should output support $(s \to t)$ at step 1 and $(a \to t)$ at step 2. The support sequence is a proof certificate: the shortest path is $s \to a \to t$. The margin at step 2 is $6 - 5 = 1$; quantization error above 0.5 may flip the predecessor, so the active-face preservation theorem gives a concrete bit-precision target.

10.2 Worked example: subset sum as max-plus reachability

For weights $w_1, \ldots, w_n$ and target T, define Boolean reachability $R_{i,s} \in \{0, -\infty\}$:

$$R_{0,0} = 0, \qquad R_{0,s} = -\infty \text{ for } s \neq 0, \tag{79}$$

$$R_{i,s} = R_{i-1,s} \oplus_{\text{trop}} R_{i-1,s-w_i}. \tag{80}$$

This is a max-plus recurrence over indicator values. A tropical head chooses skip or take. The support path traces the selected subset. A useful loss is not only $\text{CE}(\hat{R}_{n,T}, R_{n,T})$ but also a traceback loss comparing selected items.

10.3 Worked example: proof dependency routing

Suppose a proof step c_t depends on either lemma ℓ_1 or lemma ℓ_2. The model scores

$$z_j = \text{compat}(c_t, \ell_j) + \text{trust}(\ell_j) + \text{recency}(\ell_j). \tag{81}$$

Tropical support chooses $j^\star = \arg\max_j z_j$. If ℓ_1 is correct but ℓ_2 is semantically similar and false, softmax may blend them. Tropical support forces a discrete dependency that can be verified. The support loss teaches grounded proof routing.

10.4 Worked example: tone control as tropical corridor selection

Let behavior states have coordinates (u, v, w): uncertainty, verbosity, warmth. A policy has chambers:

$$C_1 = \{u \leq 0.3, v \leq 0.5\} : \text{concise direct answer}, \tag{82}$$

$$C_2 = \{u > 0.3, v \leq 0.7\} : \text{qualified explanation}, \tag{83}$$

$$C_3 = \{\text{safety risk high}\} : \text{refusal with safe alternative}. \tag{84}$$

A tropical head selects the chamber by max score. The behavior-corridor loss penalizes selecting a chamber inconsistent with evidence and safety metadata. This makes tone and personality control a routing problem with explicit supports rather than an uninspected latent drift.

11 Implementation and reproducibility

11.1 Kernel choices

The official Tropical Attention repository offers a PyTorch implementation and an optional tropical GEMM backend for max-plus products when available [3]. A reproducible TropicalGT implementation should support:

1. pure PyTorch reference kernels;
2. optional CUDA or SIMD max-plus kernels;
3. deterministic support tie-breaking;
4. exact CPU audits on small samples;
5. fake quantization during training;
6. blockwise scans for long context;
7. logging of support ids, margins, and cell ids.

11.2 Numerical issues

Tropical heads are stable in one sense and brittle in another. Output values are Lipschitz with respect to candidate perturbations, but active supports can flip near walls. This is a feature: walls are exactly where the decision is ambiguous. Implementation should therefore log both output error and support error. A model that gets numeric values right but support wrong may be unsuitable for proof or retrieval grounding.

Tie-breaking must be deterministic. Recommended rule:

$$\arg\overset{\text{lex}}{\max}_j z_j = \min\{j : z_j = \max_k z_k\}. \tag{85}$$

For graph data, lexicographic token order must be graph-canonical, not file-order dependent.

11.3 Quantization

Let b be the number of quantization bits and Δ the quantization step. If candidate values are quantized with error at most $\Delta/2$, then support is preserved whenever margin $\gamma > \Delta$. More generally, if scores and values are independently quantized with errors $\varepsilon_S, \varepsilon_V$, support is preserved when $\gamma > 2(\varepsilon_S + \varepsilon_V)$. Thus TropicalGT should report the margin distribution before export. Low-margin heads either need higher precision or must stay teacher-side.

Theorem 11.1 (Quantized support preservation)**.** *Let a tropical head have candidate values z_j and quantized values $\hat{z}_j$ satisfying $|z_j - \hat{z}_j| \le \varepsilon$ for all j. If the unique top-two margin is $\gamma > 2\varepsilon$, then* $\arg\max_j \hat{z}_j = \arg\max_j z_j$.

Proof. This is the active-support stability theorem with $\varepsilon_S + \varepsilon_V = \varepsilon$. The top candidate loses at most ε and every competitor gains at most ε, so the margin remains positive. □

11.4 Data protocols

TropicalGT should be evaluated on:

- canonical combinatorial tasks: Floyd-Warshall, Quickselect, 3SUM, subset sum, balanced partition, convex hull, 0/1 knapsack, fractional knapsack, SCC, bin packing, and minimum coin change, following the Tropical Attention suite;

- CLRS-style graph algorithm traces;
- theorem/proof dependency graphs;
- retrieval-heavy multi-document QA with provenance labels;
- long-context fill-in tasks with held-out answer bytes;
- behavior-corridor prompts requiring calibrated tone and uncertainty;
- mixed natural-language and algorithmic traces where tropical heads should activate only on algorithmic spans.

For each dataset, record train length, test length, train value range, test value range, perturbation/noise regime, exact certificate source, and license. OOD tests must not leak into training.

11.5 Ablation ladder

Every result should compare:

1. softmax transformer baseline;
2. adaptive softmax or sharpened softmax baseline;
3. full tropical replacement;
4. typed hybrid head bank;
5. hybrid plus support supervision;
6. hybrid plus margins and Bellman residual;
7. teacher tropical certificates;
8. distilled compact support codes;
9. quantized deployable model.

Report task accuracy, value error, length OOD, value OOD, perturbation OOD, support accuracy, margin distribution, active-cell entropy, inference time, parameter count, artifact bytes, and held-out bits-per-byte.

11.6 Reproducibility checklist

A release should include:

- exact data-generation scripts and seeds;
- train/validation/test split hashes;
- model configs, head types, precision, and masks;
- kernel backend and version;
- deterministic tie-breaking mode;
- exact certificate generator versions;
- ablation scripts;
- validation bits-per-byte scorer;
- support/margin log format;
- artifact byte accounting script.

12 Theoretical extensions beyond the first iteration

12.1 Hybrid softmax-tropical approximation

A concern is that a purely tropical model may underperform on diffuse semantic tasks. The following theorem explains why a hybrid bank is not less expressive on bounded domains.

Theorem 12.1 (Hybrid head dominance on bounded domains)**.** *Let $\mathcal{F}_{\text{soft}}$ be functions representable by a fixed softmax transformer family on a compact domain, and $\mathcal{F}_{\text{trop}}$ functions representable by a fixed tropical transformer family. A hybrid family with a learned gate and both head types contains $\mathcal{F}_{\text{soft}} \cup \mathcal{F}_{\text{trop}}$. If its feed-forward sublayers are universal on the compact token domain, it can approximate finite sums of softmax-representable and tropical-representable components.*

Proof. Set the gate identically 0 to recover the softmax family and identically 1 to recover the tropical family. For finite sums, route one subset of heads to softmax components and another to tropical components, then use the output projection and feed-forward sublayer to approximate the desired continuous combination on the compact domain. □

12.2 Tropical softmax limit

Softmax is related to max by the log-sum-exp limit:

$$\tau \log \sum_j \exp(z_j/\tau) \to \max_j z_j \quad \text{as } \tau \downarrow 0. \tag{86}$$

But this limit is not the same as using softmax as a stable reasoning primitive. Gradients and finite-temperature mixtures can blur support.

Lemma 12.2 (Log-sum-exp error bound)**.** *For $z \in \mathbb{R}^n$,*

$$\max_j z_j \le \tau \log \sum_j e^{z_j/\tau} \le \max_j z_j + \tau \log n. \tag{87}$$

Proof. Let $M = \max_j z_j$. Then $e^{M/\tau} \le \sum_j e^{z_j/\tau} \le n e^{M/\tau}$. Taking $\tau \log$ gives the result. □

The error grows with $\tau \log n$. Thus fixed-temperature softmax can become increasingly blurry as length grows. Tropical attention removes the entropy term rather than tuning it.

12.3 Sparse tropical kernels

Full tropical attention costs $O(L^2 d)$ if applied densely. Sparse kernels restrict key sets $\mathcal{N}(i)$. If the true support is contained in $\mathcal{N}(i)$, sparse tropical attention is exact.

Proposition 12.3 (Sparse support exactness)**.** *Let A_{ic} be the dense active support. If $A_{ic} \subseteq \mathcal{N}(i)$, then sparse tropical attention over $\mathcal{N}(i)$ returns the same output and support as dense tropical attention for (i, c).*

Proof. The maximum over all keys is attained inside $\mathcal{N}(i)$. Removing non-active keys cannot change the maximum or its active set. □

This yields an implementation strategy: train a cheap projected retrieval head to propose $\mathcal{N}(i)$, then audit support recall. The tropical head is exact conditional on recall.

12.4 Tropical retrieval theorem

Let q be a query and K_j, V_j database keys and values. Suppose a projection P approximates Hilbert distances:

$$|d_H(q, K_j) - d_H(Pq, PK_j)| \leq \varepsilon \quad \text{for all } j. \tag{88}$$

If the nearest key has margin $> 2\varepsilon$, projected tropical retrieval returns the same support.

Proof. The retrieval score is $-d_H$. Distance approximation changes every score by at most ε. Apply active-support stability. □

This theorem is useful for vector databases. A learned projection can make search efficient, but the system should report margin-certified support preservation.

13 Applications

13.1 Algorithmic reasoning

TropicalGT is most naturally suited to tasks where the target computation is a max/min recurrence, a shortest-path closure, an order statistic, a dynamic-programming table, or a greedy choice with exchange proof. It should be evaluated first on clean synthetic tasks with exact certificates and then on noisy natural variants.

13.2 Graph reasoning and knowledge graphs

A knowledge graph query often asks for the best path, most reliable evidence chain, strongest contradiction, or minimal-hop explanation. Tropical heads are appropriate for path and support selection. Softmax heads can still summarize multiple semantically related neighbors. A graph-token model can combine both.

13.3 MCP and tool reasoning

Tool calls produce typed records: schema, arguments, output, permission, timestamp, and provenance. Tropical support can choose the decisive tool output for a reasoning step. A certificate says: this answer depended on this tool result, not merely on latent memory.

13.4 Long-context reasoning

In 1–10M token contexts, dense softmax is not the intended primitive. Tropical ring scans can identify maximum-support blocks exactly over shards. The model can then unpack only a small number of high-support blocks. This is a path toward long-context efficiency: use tropical summaries for hard evidence, then local softmax for detailed reading.

13.5 Behavior and personality control

Tone, verbosity, uncertainty, compliance, and safety can be represented as behavior coordinates. Tropical routing selects behavior chambers based on evidence, task type, and safety metadata. This is not a replacement for policy or safety evaluation; it is an audit layer that records which context or rule dominated a behavioral choice.

14 Experimental plan

14.1 Hypotheses

TropicalGT makes five empirical hypotheses:

1. tropical heads improve length and value OOD on algorithmic tasks;
2. support supervision improves reasoning faithfulness and verifier agreement;
3. margin regularization improves quantized deployment and perturbation robustness;
4. hybrid head banks outperform full tropical replacement on mixed natural/algo tasks;
5. compact support codes can improve held-out bits-per-byte when they are prefix-visible and artifact-cheap.

14.2 Metrics

Report:

- held-out bits-per-byte;
- in-distribution task accuracy or MSE;
- length OOD, value OOD, perturbation OOD;
- support accuracy and support recall;
- top-two margin quantiles;
- active-cell entropy;
- proof/verifier agreement;
- retrieval support precision/recall;
- behavior-corridor violation rate;
- inference time and memory;
- artifact bytes after export.

14.3 Statistical discipline

Use matched seeds, paired bootstrap confidence intervals, and an ablation ladder. A structural metric is not a success criterion alone. For example, high support accuracy is good only if it transfers to task accuracy, verifier agreement, bits-per-byte, or controllability.

15 Risks and limitations

TropicalGT has clear risks. First, hard support can be wrong. A wrong hard support may be worse than a soft mixture. This is why support losses require exact labels or strong teacher certificates. Second, tropical kernels can be slower than optimized softmax kernels unless sparse or fused. Third, tropical geometry may encourage overdiscretization of genuinely ambiguous semantic tasks. Fourth, behavior-corridor routing may be misused if treated as a complete safety system. Fifth, mathematical certificates can become ornamental unless disciplined by bits-per-byte.

The remedy is not to optimize every objective. The remedy is staged activation, exact audit, matched ablation, and compression-aware deployment. Tropical structure should stay teacher-side unless it proves utility.

16 TropicalGT-I implementation update: graph-token reasoning agent

The first implementation pass instantiates the research program above as a compact graph-conditioned reasoning model rather than as a full contest submission. The implementation target is `TropicalGT-I`: a TokenGT-style graph tokenizer, a byte-level reasoning language model, a tropical ring attention graph encoder, a GFlowNet trajectory auxiliary, a GraphCG latent-direction auxiliary, and Plotly-based trajectory audits. This section records the precise methodology so that the paper, code, and reproducibility contract remain synchronized.

16.1 TokenGT-style graph tokenization

Each normalized training row is represented as a graph record

$$r = (\mathrm{id}, x_{\mathrm{text}}, q, a, s, G),$$

where q is a question, a an answer, s a reasoning trace, and $G = (V, E)$ a typed directed graph. If the source row lacks `graph_json`, the implementation constructs a conservative graph with a problem node, reasoning-step nodes, an answer node, `depends_on` edges, and a final `supports_answer` edge. The TokenGT tokenizer follows Kim et al. [53]: it emits a graph token, one token for each vertex, and one token for each valid edge. Vertex tokens carry endpoint identifiers (i, i); edge tokens carry (i, j); type identifiers distinguish graph, vertex, and edge tokens. The deterministic identifier map is a finite orthonormal-style coordinate chart with hashed fallback, which is sufficient for the smoke-scale model and preserves the intended equality/incidence tests. The Parameter-Golf adapter keeps the default language-model path unchanged but accepts optional graph tuples in either three-tensor form (`features`, `type_ids`, `mask`) or four-tensor form (`features`, `type_ids`, `endpoint_ids`, `mask`), with -1 endpoint ids interpreted as padding.

Sequential text is also treated as graph-structured data. For every record, TropicalGT appends a bounded derived path graph whose vertices are UTF-8 text chunks and whose edges encode the original order. This path is deterministic from the byte stream, so it is not charged as external graph side information in compression accounting. Its purpose is architectural: the byte stream and the explicit graph both enter the same graph-token interface, allowing ablations to compare plain text, text-as-path, source graph, and source graph plus text path under identical TokenGT tropical attention.

16.2 Tropical ring attention and certificates

Given graph-token states $z_1, \ldots, z_N \in \mathbb{R}^d$, the implemented tropical head computes projected queries, keys, and values and scores token compatibility by negative tropical Hilbert spread,

$$S_{ij} = -d^{-1/2} \left(\max_a (q_{ia} - k_{ja}) - \min_a (q_{ia} - k_{ja}) \right).$$

The graph context is then a max-plus reduction

$$c_{ia} = \max_j \{ S_{ij} + v_{ja} \},$$

followed by a learned linear output map. The head returns the active support $\arg\max_j S_{ij}$, the top-two margin, and support entropy. Blockwise evaluation is exact because the maximum over context shards equals the maximum over their maxima. The v1 implementation logs these quantities as certificate diagnostics: masked support entropy, soft support entropy used by the entropy regularizer, margin mean/min/p05, margin-threshold wall-hit rate, support-boundary rate, structural edge-certificate agreement, and graph/node/edge token counts.

16.3 Embedding-space Graph-of-Thought GFlowNet training

Graph-of-Thought reasoning [58] is internalized as a stochastic process over pooled graph-token states. A lightweight policy $\pi_\theta(a \mid h_t)$ samples a small discrete action vocabulary representing expand, merge, refine, stop, retrieve, verify, compress, and reject moves. The first implementation uses trajectory balance from the continuous GFlowNet theory of Lahlou et al. [59],

$$\mathcal{L}_{\mathrm{TB}} = \mathbb{E}_\tau \left[\left(\log Z_\theta + \sum_t \log \pi_\theta(a_t \mid h_t) - \log R(\tau) \right)^2 \right],$$

with a smoke-stage reward derived from likelihood improvement and tropical-margin stability. Later runs can replace this proxy with verifier scores, answer exactness, retrieval provenance, and certificate validity without changing the interface.

At inference time the same action vocabulary defines a bounded test-time scaling controller. Starting from the prompt graph, TropicalGT scores a frontier of graph-of-thought candidates by a deployment proxy

$$S(\tau) = -\,\mathrm{NLL}(\tau) + \lambda_m \bar{m}(\tau) + \lambda_\pi \max_a \pi_\theta(a \mid h_\tau) - \lambda_c |T_\tau|,$$

where $\bar{m}(\tau)$ is the mean tropical top-two margin and $|T_\tau|$ is the graph-token budget. The controller expands only the top w candidates for depth d, using the highest-probability GFlowNet actions as branches. This does not assert that the smoke model has learned a strong verifier; rather, it makes the scaling rule explicit and auditable. Each candidate records its action path, NLL proxy, tropical support trace, GFlowNet action probabilities, GraphCG direction diagnostics, token budget, and filtered simplicial object, so longer experiments can swap in stronger rewards without changing the report format.

16.4 GraphCG latent steering

The GraphCG auxiliary follows Liu et al. [60]: learned directions $d_1, \ldots, d_m$ define steerable latent edits in pooled graph state space. The implementation uses a same-condition contrastive term over direction-shifted states, plus orthogonality, sparsity, and an explicit full-rank singular-value barrier. If $D \in \mathbb{R}^{m \times d}$ is the row-normalized direction matrix and $r = \min(m, d)$, the full-rank term penalizes

$$\frac{1}{r} \sum_{i=1}^{r} \max\{0, \eta - \sigma_i(D)\}^2,$$

where $\eta > 0$ is a small rank margin and $\sigma_i(D)$ are singular values. Training and inference reports therefore include effective rank, numerical rank, minimum and maximum singular values, and condition proxies in addition to the Gram matrix. This regularizer is intentionally light: it should organize the latent chart in which graph-of-thought trajectories move, not dominate the byte/reasoning objective. Its weight and direction count are promoted only when matched ablations improve held-out BPB or graph-BPB.

16.5 Multiparameter persistence, derived signatures, and analogical memory

The current audit layer attaches a finite filtered simplicial object to each reasoning step and to the cumulative graph-of-thought trajectory. Vertices represent graph or reasoning states, edges represent incidence or parent-child transitions, and two-simplices represent directed length-two reasoning paths. For a bounded complex K, the implementation computes chain groups over $\mathbb{F}_2$, boundary ranks, Betti vectors, Euler characteristic, scalar persistent homology when a backend such as Gudhi is present,

Stanley–Reisner degree-two nonfaces, Taylor-resolution upper bounds, DG-commutative cochain ranks, cup-product support counts, and a compact derived-equivalence signature.

For multiparameter persistence, the v1 implementation follows the algebraic dictionary used in multiparameter persistent homology [35]: a finite n-parameter persistence module is represented as a multigraded module over $\mathbb{F}_2[x_1, \ldots, x_n]$. The implemented bounded report uses three parameters: filtration value, simplex dimension, and vertex-position scale. It records multigraded generators for chain modules, monomial-labeled boundary maps, a finite grid of fiber ranks, sampled comparable-pair rank invariants, and a free-resolution proxy. If a dedicated package such as `multipers` [36] is installed, it can be used as a backend for richer signed-measure or invariant computations; otherwise the repo uses this exact bounded fallback. This is enough for smoke-scale audits and for training-time correlations: each algebraic summary can be tested against BPB, graph-BPB, verifier score, and reward stability.

The analogical-reasoning component generalizes the categorical analogy perspective studied in transformer reasoning [37]. A memory item is not merely a vector. It is a packet

$$M = (e, \Sigma_K, \rho_K, \partial_K, \mu_K, \Gamma_K),$$

where e is a pooled embedding, Σ_K is the trajectory graph on reasoning states, ρ_K is the filtered simplicial object, ∂_K is the chain differential, μ_K records selected multiplicative or cup-product data, and Γ_K is a derived signature containing Betti, persistence, rank-invariant, and graph statistics. Retrieval scores combine embedding cosine similarity, signature cosine similarity, and a quality score derived from reward and NLL. Thus memory retrieval is analogical in two senses: latent vectors must be nearby, and the algebraic-topological shape of the reasoning trajectory must be comparable. The trainable memory head projects pooled graph states into the query space; its contrastive loss is ablated separately and is promoted only if it improves ordinary BPB or graph-BPB.

16.6 Bits-per-byte and graph bits-per-byte

Because the parameter-golf baseline is byte-level, ordinary BPB is computed exactly from the mean natural-log NLL over non-padding byte targets:

$$\mathrm{BPB}_{\mathrm{text}} = \frac{\sum_t -\log p_\theta(b_t \mid b_{<t}, G)}{\log 2 \cdot |\{t : b_t \neq \mathrm{pad}\}|}.$$

TropicalGT additionally reports graph-aware scores. Let B be the number of predicted target bytes, $S(G)$ the deterministic structural byte count of the actually tokenized graph tokens, and $J(G)$ the explicit JSON byte count of non-derived graph side information after removing the deterministic text path. Here $S(G)$ is computed from live graph-token masks, node counts, and edge endpoint ids when endpoint tensors are supplied, so the graph-BPB denominator reflects the same TokenGT structure that conditions the model. With side-information weight ρ, the implementation logs

$$\mathrm{BPB}_{\mathrm{graph}} = \frac{\mathrm{NLL}_{\mathrm{bits}} + 8\rho J(G)}{B + S(G)}, \qquad \mathrm{BPB}_{\mathrm{side}} = \frac{\mathrm{NLL}_{\mathrm{bits}} + 8\rho J(G)}{B}.$$

It also logs the optimistic conditioning score $\mathrm{NLL}_{\mathrm{bits}}/(B + S(G))$, which treats graph structure as already present in context. These three numbers separate contest-style byte compression, graph-conditioned compression, and honest side-information accounting. All tropical, GraphCG, GFlowNet, persistence, and analogical-memory metrics are therefore evaluated by whether they correlate with, or causally improve under ablation, $\mathrm{BPB}_{\mathrm{text}}$ and $\mathrm{BPB}_{\mathrm{graph}}$. The stripped Parameter-Golf export path keeps this accounting separate from the research stack: it packages only the baseline training script, the TokenGT graph adapter, the compressed model artifact, and a manifest that checks code-plus-artifact bytes against the local 16,000,000-byte cap and records the BPB/graph-BPB contract.

16.7 Training, evaluation, inference, and visualization contract

The runnable contract is implemented by eight scripts: `train_tropicalgt_i.py`, `eval_tropicalgt_i.py`, `infer_tropicalgt_i.py`, `validate_tropicalgt_i.py`, `render_reasoning_visualizations.py`, `analyze_bpb_ablations.py`, `run_bpb_ablation_grid.py`, and `audit_tropicalgt_i_readiness.py`. Data-backed configurations require the moved ToricGT parquet shards explicitly: the loader first builds a row-group metadata index for the train, validation, and test splits, reads examples through a bounded row-group cache rather than concatenating the full corpus into memory, and refuses to fall back to fixtures when `require_data` is enabled. Long-run configurations use a row-group-aware chunk-shuffle sampler that randomizes parquet row-group order while preserving row-group-local reads, avoiding the cache thrashing that would be induced by full random access over multi-million-row shard collections. The validation script emits this parquet manifest together with graph-token statistics, graph-json fallback rate, node/edge statistics, and sample graph-token descriptors before long training begins. The readiness audit script combines environment/package checks, required-data manifest checks, sample TokenGT conversion, deterministic sequential-text graphification, paper artifact checks, checkpoint reload, finite evaluation, text-BPB and graph-BPB gates, bounded inference scaling, optional topological algebra audits, ablation-tool availability, and optional visualization generation into a single JSON/Markdown proof bundle. For a full training configuration with no checkpoint yet, the same audit can run a bounded dry-run optimizer step on CUDA: it builds the configured model, samples moved parquet records, executes forward/backward/update, verifies the W&B key path, and gates finite loss, gradient norm, text BPB, and graph-BPB before the long job is launched. Training reports language-model NLL, exact text BPB, graph-BPB, graph side-information BPB, GFlowNet trajectory-balance loss, GraphCG contrastive and full-rank direction geometry terms, finite graph-certificate loss and agreement, tropical support entropy, tropical margins, wall-hit rate, graph-token statistics, graph JSON fallback rate, graph JSON sequentialization rate, multiparameter persistence summaries, analogical-memory query norms, examples/sec, tokens/sec, GPU memory, and step throughput to W&B when enabled. The ablation analyzer consumes one or more training reports and ranks logged metrics by correlation with $\mathrm{BPB}_{\mathrm{text}}$, $\mathrm{BPB}_{\mathrm{graph}}$, and held-out evaluation variants; the ablation-grid runner generates same-seed loss-weight variants such as no-GraphCG, no-GFlowNet, no-certificate, no-tropical-regularizer, and no-auxiliary, then optionally trains them and runs the analyzer. These reports are screening devices for matched ablations, not causal evidence by themselves. The first finite certificate objective is deliberately modest: an edge token is structurally certified when its active tropical support is the edge token itself or one of its endpoint vertex tokens. This is a TokenGT skeleton certificate rather than a semantic proof; it should be promoted only when it improves held-out BPB, verifier score, support faithfulness, or downstream reasoning metrics. Checkpoints store model state, optimizer state, step, metric history, config, and random-number-generator state; long runs can resume from either the final checkpoint or a periodic `.latest.pt` checkpoint. Evaluation reports aggregate NLL/BPB/graph-BPB together with optional per-record NLL, tropical support histograms, graph-token traces, filtered simplicial objects, and opt-in topological algebra audits. Inference emits the generated byte-level argmax string plus BPB accounting, tropical support/margin diagnostics, GFlowNet action probabilities, GraphCG direction summaries, a graph-token trace, the filtered object for the prompt graph, optional multiparameter algebra, optional analogical-memory retrieval, and an optional bounded inference-scaling report containing all expanded candidates and the selected graph-of-thought path. The visualization script projects graph-of-thought candidate embeddings by PCA and writes dark-mode interactive three-dimensional trajectories whose edges are the reasoning graph. Hovering over a node updates a rendered SVG side panel for that node's filtered simplicial object, so the audit shows actual vertices, edges, two-simplices, and filtration colors rather than only textual summaries. The same HTML stores the complete object in JSON, and

the PCA/NLL view adds a smoothed heatmapped NLL surface under the reasoning graph or as the NLL-height field. Additional Plotly artifacts visualize tropical support heatmaps, persistence barcodes, persistence-module Betti profiles, GraphCG direction cosines, GraphCG full-rank singular spectra, GraphCG training direction geometry, analogical memory retrieval scores, and training metrics. In the first implementation each filtered object contains vertices as 0-simplices, graph edges as 1-simplices, directed length-two reasoning paths as 2-simplices, filtration thresholds, and a compact summary; longer runs can replace the deterministic smoke filtration with learned margin, verifier, or persistence-based weights.

This implementation deliberately omits toric-only mechanisms from the ToricGT paper unless they admit a tropical meaning. Ported methods are the dataset schema, graph-of-thought trajectory view, GFlowNet branch training, GraphCG steering, W&B discipline, reproducibility reports, and latent-trajectory visual audits. Omitted methods include noncommutative toric phase memory, toric ideals, and torus-specific chamber constraints; tropical replacements are active faces, Hilbert margins, max-plus supports, and fan-cell certificates.

17 Conclusion

Tropical Attention showed that replacing softmax with max-plus projective attention can restore sharp, scale-invariant, polyhedral reasoning for combinatorial tasks. TropicalGT extends this insight into a broader reasoning-agent methodology. It treats tropical heads as auditable dynamic-programming primitives, active supports as finite certificates, margins as stability witnesses, Newton polytopes as decision geometry, graph tokens as equivariant carriers, and bits-per-byte as the deployment arbiter. The resulting program is both mathematical and pragmatic: prove exact statements for finite structures; train with explicit support, margin, recurrence, and certificate losses; and keep only what improves held-out compression or reasoning quality.

A Appendix A: additional proofs

A.1 Proof of finite tropical circuit simulation

Definition A.1 (Layered tropical circuit)**.** *A layered tropical circuit is a finite directed acyclic graph whose input gates are variables or constants and whose internal gates compute either* $u \oplus_{\mathrm{trop}} v = \max(u, v)$ *or* $u \odot_{\mathrm{trop}} v = u + v$*. Edges go from layer* ℓ *to layer* $\ell + 1$*.*

Theorem A.2 (TropicalGT realizes finite layered tropical circuits)**.** *Let* C *be a layered tropical circuit of depth* L*, width at most* W*, and real constants in a bounded set. There is a TropicalGT stack of depth* $L + O(1)$*, width polynomial in* W*, and tropical heads polynomial in* W*, that computes the same circuit on bounded inputs in exact real arithmetic.*

Proof. Represent each circuit gate at layer ℓ by a token channel. A sum gate $u + v + c$ is computed by an affine feed-forward map because ordinary addition is available in the Euclidean residual stream and also equals tropical multiplication. A max gate $\max(u + c_1, v + c_2)$ is computed by a tropical head with two candidate keys whose values are $u + c_1$ and $v + c_2$. Gates in the same layer are independent and can be computed in parallel by separate channels or heads. Proceed by induction over layers. The input layer is represented exactly by token values. If layer ℓ is represented exactly, the above constructions represent every gate in layer $\ell + 1$ exactly. After L steps the outputs are represented exactly. □

A.2 Approximate version under smooth devaluation

If the tropical output is passed through a smooth devaluation map δ, exact equality is lost. On a compact domain, any fixed smooth map is Lipschitz. Thus if tropical computation is ε-accurate before devaluation, the Euclidean output is $L_\delta \varepsilon$-accurate after devaluation. This gives an immediate approximation theorem for practical architectures.

B Appendix B: implementation recipes

B.1 Tropical Hilbert distance kernel

```
def hilbert_distance(Q, K):
    # Q: [B,H,Lq,D], K: [B,H,Lk,D]
    diff = Q[:, :, :, None, :] - K[:, :, None, :, :]
    return diff.max(-1).values - diff.min(-1).values
```

B.2 Support and margin logging

```
def support_and_margin(scores):
    # scores: [B,H,Lq,Lk,C] or [B,H,Lq,Lk]
    top2 = torch.topk(scores, k=2, dim=-1).values
    support = torch.argmax(scores, dim=-1)
    margin = top2[..., 0] - top2[..., 1]
    return support, margin
```

B.3 Fake quantization audit

```
def fake_quantize(x, bits=8, clip=6.0):
    levels = 2 ** bits - 1
    x = x.clamp(-clip, clip)
    scale = levels / (2 * clip)
    q = torch.round((x + clip) * scale) / scale - clip
    return q

with torch.no_grad():
    y, supp = tropical_head(X, Wq, Wk, Wv, mask)
    yq, suppq = tropical_head(fake_quantize(X), fake_quantize(Wq),
                              fake_quantize(Wk), fake_quantize(Wv), mask)
    support_flip_rate = (supp != suppq).float().mean()
    value_error = (y - yq).abs().mean()
```

C Appendix C: objective card table

ID	Objective	Main signal	Deployment rule
1	Support supervision	active predecessor/evidence agreement	keep only if support code improves bytes or verifier
2	Margin regularization	stability to quantization/noise	requires margin audit
3	Fan-cell consistency	OOD invariance	deploy as cell code only if compressed
4	Hilbert contrastive	projective retrieval geometry	compare against cosine retrieval
5	Bellman residual	recurrence correctness	teacher-side unless distilled
6	Closure distillation	all-pairs/path closure	compact closure code only
7	Active path	predecessor proof	useful for citation/proof tasks
8	Polynomial sparsity	small candidate dictionary	improves artifact size
9	Cell entropy	avoid collapse/memorization	metric, rarely loss in deployment
10	Annealing	soft-to-hard curriculum	training-only
11	Robustness	perturbation stability	deploy if value OOD improves
12	Shift invariance	projective consistency	cheap augmentation
13	Retrieval support	evidence precision/recall	deploy if retrieval improves
14	Flow conservation	graph-of-thought consistency	verifier-facing
15	Contradiction barrier	incompatible supports	safety/evidence audit
16	Bits-per-byte gate	compression utility	mandatory
17	Proof verifier	support entails claim	proof tasks only
18	Stopping support	minimal context length	long-context deployment
19	Quantized faces	export stability	mandatory for quantized heads
20	Behavior routing	tone/safety chamber	deploy only with safety audits
21	Polytope volume	tied alternative control	task-specific
22	Tableau certificate	rank/sorting combinatorics	useful for order tasks
23	Memory recurrence	max-plus memory update	retrieval/memory tasks
24	Circuit description length	simple tropical circuits	model selection

D Appendix D: reproducible experimental command sketch

```
# 1. Generate deterministic data
python scripts/generate_task.py --task floyd_warshall --n_train 8 \
  --n_test 16 32 64 --value_range 0 10 --seed 1 --out data/fw

# 2. Train baseline
python train.py --config configs/softmax.yaml --data data/fw --seed 1

# 3. Train tropical head model
python train.py --config configs/tropicalgt.yaml --data data/fw --seed 1 \
  --log_supports --log_margins

# 4. Run exact audit
python audit.py --ckpt runs/tropicalgt/seed1.pt --data data/fw/test_64 \
  --metrics task support margin value_ood length_ood perturbation

# 5. Quantize and score held-out bytes
python export_quantized.py --ckpt runs/tropicalgt/seed1.pt --bits 8
python score_bits_per_byte.py --artifact export/model.bin --heldout heldout.bytes
```

E Appendix E: extended TropicalGT objective cards

This appendix gives implementation-facing objective cards. They are intentionally more operational than the main text. Each objective begins teacher-side, runs as an audit, and is promoted only when held-out bits-per-byte, verifier accuracy, support recall, context efficiency, or behavior-control metrics improve under matched ablations.

E.1 Objective 25: tropical support conditional information

Let A_t be the active support certificate of a tropical head at time t, and let B_{t+1} be the next byte or graph token. Define

$$U_{\text{supp}} = I(B_{t+1}; A_t \mid \mathcal{F}_t) - \lambda \,\text{bits}(A_t). \tag{89}$$

The empirical estimator uses two predictors, one with and one without the support certificate:

$$\widehat{I} = \frac{1}{T}\sum_t \log_2 q_\phi(B_{t+1} \mid \mathcal{F}_t, A_t) - \log_2 q_\psi(B_{t+1} \mid \mathcal{F}_t). \tag{90}$$

The goal is not high support entropy. High support entropy can indicate unstable reasoning. The goal is support information that predicts future bytes, verifier outcomes, or valid reasoning moves. In graph tasks this separates useful active predecessors from decorative attention visualizations.

Proposition E.1 (Support utility decomposition)**.** *If A_t is prefix-visible and both predictors are Bayes optimal, then the expected log-score improvement from revealing A_t equals $I(B_{t+1}; A_t \mid \mathcal{F}_t)$ bits.*

Proof. The Bayes expected negative log loss without A_t is $H(B_{t+1} \mid \mathcal{F}_t)$. With A_t it is $H(B_{t+1} \mid \mathcal{F}_t, A_t)$. Their difference is the conditional mutual information. □

E.2 Objective 26: min-plus verifier distance

Given a verifier graph whose nodes are proof obligations and whose edges are admissible proof refinements, let $d_t(v)$ be the min-plus distance from the current reasoning state to obligation v. A TropicalGT verifier head predicts

$$\hat{d}_{t+1}(v) = \min\left(\hat{d}_t(v), \min_{u \to v}\{\hat{d}_t(u) + \hat{w}(u,v)\}\right). \tag{91}$$

Train with

$$\mathcal{L}_{\text{vdist}} = \sum_v \rho_v |\hat{d}_T(v) - d^\star_T(v)| + \lambda \sum_t \text{TV}(\hat{d}_{t+1} - \hat{d}_t). \tag{92}$$

The distance target can be generated from exact proof graphs, rubric decompositions, or oracle trajectory classifiers. Compression improves when proof obligations become reusable dynamic-programming states rather than repeated natural-language reminders.

E.3 Objective 27: tropical consistency under graph contraction

Let $\kappa : G \to G/H$ contract a subgraph H whose internal computation has been summarized by a tropical state vector s_H. A contraction-consistent head satisfies

$$F_\theta(G) \simeq \kappa^* F_\theta(G/H, s_H). \tag{93}$$

Use

$$\mathcal{L}_{\text{contr}} = \|P_\kappa F_\theta(G) - F_\theta(G/H, s_H)\|_2^2 + \beta \,\text{bits}(s_H). \tag{94}$$

This is a direct training rule for hierarchical reasoning. If a subproof, tool trace, or retrieved document can be summarized without changing the answer distribution, the context window can use the summary and save tokens.

E.4 Objective 28: Hilbert-metric calibration

The Hilbert projective metric is scale-invariant. For two keys k_i, k_j, let $d_H(k_i, k_j)$ denote their tropical projective distance. Suppose teacher supports say that two states are equivalent up to tropical scaling. Train

$$\mathcal{L}_H = \sum_{(i,j)\in P^+} d_H(k_i, k_j)^2 + \sum_{(i,j)\in P^-} \max(0, \gamma - d_H(k_i, k_j))^2. \tag{95}$$

This objective helps when absolute logit scale is arbitrary but relative tropical order matters. It is especially useful for length extrapolation because dynamic-programming values often grow with sequence length while their projective differences remain stable.

E.5 Objective 29: support-margin early stopping

A reasoning trajectory should stop when additional context is unlikely to change the active tropical support or verifier outcome. Define the support margin

$$\Gamma_t = \min_{i,c} \big(z^{(1)}_{ic,t} - z^{(2)}_{ic,t}\big) \tag{96}$$

and predicted answer entropy H_t. Train a stopping head with

$$\mathcal{L}_{\text{stop}} = \mathrm{CE}(\hat{s}_t, s^\star_t) + \lambda \max(0, \gamma - \Gamma_t)\hat{s}_t + \mu H_t \hat{s}_t. \tag{97}$$

Stopping with low margin is penalized. Continuing after high-margin, low-entropy convergence is penalized by context-length cost. This connects tropical stability to efficient bits-per-byte deployment.

E.6 Objective 30: tropical beam diversity with exact merge

Let a trajectory sampler produce beams $\tau^1, \ldots, \tau^K$. Each beam has a tropical support path $A(\tau^k)$ and score R_k. Diversity is measured by support-path distance,

$$D_{kl} = 1 - \frac{|A(\tau^k) \cap A(\tau^l)|}{|A(\tau^k) \cup A(\tau^l)| + \varepsilon}. \tag{98}$$

Use

$$\mathcal{L}_{\text{beam}} = -\operatorname{LSE}_k R_k + \lambda \sum_{k<l} \max(0, \delta - D_{kl})^2. \tag{99}$$

This encourages multiple genuinely different proof or retrieval routes. Exact merge is possible when two beams reach the same tropical state vector because future max-plus recurrence depends only on that state.

E.7 Objective 31: tropical monotonicity for cumulative evidence

For verified evidence accumulation, some coordinates should be monotone under additional reliable evidence. Let e_t be a coordinate for verified support. Require

$$\mathcal{L}_{\text{mono}} = \sum_t \max(0, e_t - e_{t+1})^2 r_t, \tag{100}$$

where r_t is zero for retractions or contradicted evidence. This improves control because a model should not forget verified evidence unless contradiction, tool invalidation, or permission boundary changes the state.

E.8 Objective 32: tropical contradiction cut

Let C be a contradiction graph on claims, with an edge $a \dashv b$ when two claims cannot both be active. If s_a, s_b are tropical support scores, train

$$\mathcal{L}_{\text{cut}} = \sum_{a \dashv b} \max(0, s_a + s_b - \eta)^2. \tag{101}$$

The loss is a max-plus analogue of a cut constraint. It is useful for retrieval-heavy agents where incompatible memories can be retrieved together.

E.9 Objective 33: parametric tropical curriculum

For a synthetic problem family indexed by length n, value scale B, and graph density p, train a curriculum distribution $\pi_\alpha(n, B, p)$. The controller increases difficulty when support recall and bits-per-byte pass thresholds:

$$\alpha_{t+1} = \alpha_t + \eta \nabla_\alpha \big[\text{Acc}_{\text{OOD}} - \lambda \, \text{bits-per-byte}_{\text{val}}\big]. \tag{102}$$

This objective separates interpolation difficulty from algorithmic extrapolation. A model is not credited for solving large instances if its active supports no longer match exact dynamic-programming supports.

E.10 Objective 34: tropical explanation compression

An explanation is a compressed certificate Z for a reasoning trajectory. Train a variational code with

$$\mathcal{L}_{\text{expl}} = -\log q_\theta(y \mid x, Z) + \lambda |Z| + \mu \, \text{CE}(\hat{A}(Z), A^\star). \tag{103}$$

The certificate may contain active predecessor ids, margins, recurrence names, and verified terminal states. It is useful only if it improves prediction or auditing at lower byte cost than a full chain-of-thought transcript.

E.11 Objective 35: tropical Jacobian sparsity

For a piecewise-linear tropical map F, the local Jacobian is a selector matrix plus affine weights on each active region. Estimate it away from ties and penalize

$$\mathcal{L}_{\text{Jac}} = \|J_F\|_{1,\text{off}} + \lambda \, \text{rank}_\tau(J_F), \tag{104}$$

where rank_τ is a soft threshold rank proxy. This encourages decisions that depend on few relevant predecessors, improving interpretability and retrieval locality.

E.12 Objective 36: tropical oracle agreement

Combine TropicalGT with oracle trajectory classifiers. Let $o_\omega(\tau)$ classify a trajectory by accuracy, tone, compliance, grounding, and efficiency. Let $A(\tau)$ be the tropical support path. Train

$$\mathcal{L}_{\text{oracle}} = \text{CE}(o_\omega(\tau), y_{\text{prop}}) + \lambda \| r_\phi(A(\tau)) - o_\omega(\tau) \|_2^2. \tag{105}$$

When support paths predict bad behavior before the final answer, the controller can intervene early by changing retrieval, asking for verification, or projecting the trajectory into a safer behavior corridor.

F Appendix F: detailed implementation protocol

F.1 Data schema

A reproducible TropicalGT record is a JSON, msgpack, or Arrow object with these fields:

1. `id`: stable hash of task instance, generator, and split;
2. `graph`: node table, edge table, masks, typed attributes, and adjacency indices;
3. `tokens`: optional natural-language prefix and byte target;
4. `recurrence`: exact tropical, min-plus, max-plus, Boolean, or log-semiring recurrence name;
5. `oracle_states`: exact dynamic-programming states for small instances or sampled states for large instances;
6. `supports`: active predecessor sets, tie sets, and margins;
7. `certificates`: optional toric, persistence, sheaf, or commutative-algebra summaries;
8. `splits`: family id, size id, value-scale id, and leakage-resistant split id.

Every field used by a deployment loss must be prefix-visible at the time it is consumed. If a field uses the target answer or future bytes, it is teacher-only and cannot be used in prequential scoring.

F.2 Reference training loop

```
for batch in loader:
    graph = tokenize_graph(batch.graph)
    prefix = tokenize_bytes(batch.prefix)
    H = model.encode(graph, prefix)
    Y, support, margin = model.tropical_heads(H, return_support=True)
    loss = bits_per_byte_loss(model, batch.target_bytes)
    certs = certificate_cache.lookup(batch.ids)
    for family in active_families:
        if controller.enabled(family):
            loss = loss + controller.weight(family) * family.loss(H, Y, support, certs)
    loss.backward()
    optimizer.step()
    if step % audit_interval == 0:
        logs = exact_support_audit(model, batch, certs)
        controller.update(logs, validation_bits_per_byte())
```

F.3 Kernel choices

The dense tropical kernel is a reference implementation, not the intended long-context kernel. Practical kernels use one of four strategies.

1. Block scan: compute exact maxima within blocks, then reduce block maxima.
2. Sparse candidate sets: use retrieval to propose $\mathcal{N}(i)$, then audit support recall.
3. Hybrid soft-tropical: reserve tropical heads for hard dynamic-programming channels and use softmax for semantic interpolation.
4. Quantized tropical: fake-quantize keys and values; promote only if active supports are stable under round trip.

Theorem F.1 (Quantized support preservation)**.** *Let $z_j = S_{ij} + V_{jc}$ have unique maximizer $j^\star$ and margin $\gamma = z_{j^\star} - \max_{j \neq j^\star} z_j$. If quantization perturbs every z_j by at most $\varepsilon < \gamma/2$, then the quantized tropical head has the same active support.*

Proof. For any $j \neq j^\star$, the perturbed gap is at least $\gamma - 2\varepsilon > 0$. Hence $j^\star$ remains strictly maximal. □

F.4 Reproducibility grid

Report at least the following grid.

Axis	Values	Reported metrics
length	train, 2x, 4x, 8x	accuracy, support recall, bits-per-byte
value scale	train, 2x, 8x	margin, verifier error, calibration
graph size	train, 2x, 4x	runtime, memory, exactness
context budget	16k, 128k, 1M, 10M	marginal utility, stopping length
quantization	fp32, bf16, int8, int4	support stability, exported bits-per-byte

G Appendix G: ablation suites and failure-mode diagnostics

G.1 Ablation philosophy

TropicalGT should be evaluated by a ladder, not by a single full-model comparison. The ladder is: dense Transformer baseline; graph-token baseline with ordinary softmax heads; hybrid model with tropical heads disabled but probes logged; tropical heads enabled on synthetic dynamic-programming channels only; tropical heads enabled on retrieval and verifier channels; support-supervised teacher; distilled student; full bits-per-byte and behavior audit. A component is credited only if it beats the immediate predecessor under matched seeds, data, training tokens, parameter budget, and artifact-byte accounting.

G.2 Support faithfulness audit

A tropical support is faithful when it identifies the actual finite certificate responsible for the output. For a shortest-path instance, the support should be the active predecessor edge. For knapsack, it should be the include/exclude predecessor cell. For retrieval, it should be a graph path to evidence or memory. Define

$$P_{\mathrm{supp}} = \frac{|A_\theta \cap A^\star|}{|A_\theta| + \varepsilon}, \qquad R_{\mathrm{supp}} = \frac{|A_\theta \cap A^\star|}{|A^\star| + \varepsilon}. \tag{106}$$

Report these separately from answer accuracy. A model can answer correctly while using an unfaithful support, especially when dataset artifacts leak shortcuts.

G.3 Length and scale extrapolation audits

For every task family, choose training lengths $n \in [n_{\min}, n_{\max}]$ and evaluate on $2n_{\max}, 4n_{\max}, 8n_{\max}$. Report

$$\Delta_{\mathrm{len}}(m) = \mathrm{Acc}(mn_{\max}) - \mathrm{Acc}(n_{\max}), \qquad m \in \{2, 4, 8\}. \tag{107}$$

For numerical dynamic-programming tasks, also increase weights or capacities while holding graph size fixed. Define the scale-regret curve

$$R_{\mathrm{scale}}(B) = \mathcal{L}(B) - \mathcal{L}(B_{\mathrm{train}}). \tag{108}$$

Report this for accuracy loss, support error, and bits-per-byte. The useful target is stable extrapolation across scales seen in practical algorithmic tasks, not infinite numeric generalization.

G.4 Retrieval audit

For memory and MCP settings, exact support labels are often unavailable. Use synthetic graph databases where the evidence path is known, and natural databases where evidence is approximated by citations, tool-call provenance, or manual labels. Report candidate recall, tropical path recall, answer grounding, and context efficiency. Low candidate recall indicates a retrieval-projection failure. High candidate recall but low tropical path recall indicates a tropical scorer failure. High path recall but low grounding indicates a decoder or verifier failure.

G.5 Behavior audit

Behavior controls should not be evaluated only by classifier scores. A tone or compliance classifier can be gamed by superficial phrasing. The TropicalGT behavior audit pairs every classifier label with a structural state: evidence strength, support margin, permission boundary, contradiction mass, and

uncertainty entropy. A calibrated model should have monotone relations between assertiveness and evidence margin, and refusal strength should increase with permission or safety boundary scores.

Failure	Symptom	Corrective action
Softmax leakage	attention mass spread over irrelevant predecessors	increase tropical head weight or margin loss; audit support recall
Tropical brittleness	support changes under tiny perturbations	add margin regularization and tie-aware labels
Decorative support	supports are stable but unrelated to exact certificate	train support conditional information; reject if utility is zero
Length overfitting	good in-distribution accuracy but poor long length	curriculum on recurrence depth and graph size
Scale failure	larger weights break recurrence	Hilbert-metric calibration and projective normalization
Retrieval overreach	many context blocks with low utility	marginal context utility gate and contraction summaries
Behavior drift	tone changes without evidence-state change	behavior-chart monotonicity and oracle agreement
Compression failure	structural module improves task but worsens bits-per-byte	keep teacher-side and distill only support summaries

H Appendix H: additional proofs and worked examples

Lemma H.1 (Maximum as a nonexpansive operator)**.** *For vectors $a, b \in \mathbb{R}^n$,*

$$|\max_i a_i - \max_i b_i| \le \|a - b\|_\infty. \tag{109}$$

Proof. For every i, $a_i \le b_i + \|a - b\|_\infty$, so $\max_i a_i \le \max_i b_i + \|a - b\|_\infty$. Reversing a and b gives the opposite inequality. □

Theorem H.2 (Tropical recurrence stability)**.** *Consider a recurrence $x_{t+1} = F_t(x_t)$ where every F_t is composed of affine maps with operator norm at most L and coordinatewise maxima. If perturbed maps $\hat{F}_t$ satisfy $\sup_x \|F_t(x) - \hat{F}_t(x)\|_\infty \le \varepsilon_t$, then*

$$\|x_T - \hat{x}_T\|_\infty \le \sum_{t=0}^{T-1} L^{T-1-t}\varepsilon_t, \tag{110}$$

assuming the same initial state.

Proof. The maximum operator is nonexpansive and the affine parts are L-Lipschitz. Thus F_t is L-Lipschitz. The error recursion is

$$\|x_{t+1} - \hat{x}_{t+1}\|_\infty \le L\|x_t - \hat{x}_t\|_\infty + \varepsilon_t. \tag{111}$$

Iterating from zero initial error gives the stated bound. □

Proposition H.3 (Tie-aware supervision is necessary)**.** *If an exact recurrence has a tie set $A^\star$ with more than one maximizer, any single-index cross-entropy target introduces arbitrary symmetry breaking. A set-valued target with uniform mass over $A^\star$ is invariant under relabeling of tied maximizers.*

Proof. A single-index target chooses one element of $A^\star$, so permuting two tied maximizers can change the label while leaving the mathematical recurrence unchanged. The uniform distribution on $A^\star$ is fixed by every permutation preserving the set. □

Theorem H.4 (Support recall implies exact sparse output)**.** *Let a dense tropical head compute $Y_{ic} = \max_{j \in J} z_{icj}$. Let $N_i \subseteq J$ be a sparse candidate set. If $A_{ic} = \arg\max_{j \in J} z_{icj} \subseteq N_i$, then the sparse head $\max_{j \in N_i} z_{icj}$ has the same value and same active support.*

Proof. The dense maximum is attained by every element of A_{ic}, and all these elements are included in N_i. No element outside A_{ic} exceeds the maximum by definition. Restricting to N_i preserves the maximum value and the maximizers. □

H.1 Shortest path with active predecessor certificate

Consider the graph $s \to a \to t$ and $s \to t$ with weights $w(s, a) = 2$, $w(a, t) = 3$, and $w(s, t) = 6$. The min-plus recurrence yields

$$d_0(s) = 0, \qquad d_0(a) = \infty, \qquad d_0(t) = \infty, \tag{112}$$

$$d_1(a) = 2, \qquad d_1(t) = 6, \tag{113}$$

$$d_2(t) = \min(6, 2 + 3) = 5. \tag{114}$$

The active predecessor for t changes from s to a at the second relaxation. A TropicalGT head should expose support certificate $A_t = \{a\}$ and margin 1. If a perturbation changes all candidate scores by less than $1/2$, the active predecessor is stable.

H.2 Subset sum as Boolean tropical reachability

For weights $2, 3, 5$ and target 7, use Boolean tropical values $R_{i,s} \in \{0, -\infty\}$, where 0 means reachable. Initialize $R_{0,0} = 0$ and $R_{0,s} = -\infty$ for $s \neq 0$. The recurrence is

$$R_{i,s} = \max(R_{i-1,s}, R_{i-1,s-w_i}). \tag{115}$$

The target 7 is reached by $2 + 5$. The active support certificate for $(i, s) = (3, 7)$ points to $(2, 2)$, then to $(1, 2)$, then to $(0, 0)$. This chain is an explanation certificate whose byte cost is logarithmic in the table size if stored as predecessor bits.

H.3 Tropical retrieval over a knowledge graph

Suppose a long-context memory graph contains claims c_i, tools u_j, and evidence nodes e_k. A question asks for a claim that depends on evidence nodes and a successful tool call. A retrieval head proposes $\mathcal{N}(q)$. The tropical verifier computes

$$S(q, c) = \max_{p: q \rightsquigarrow c} \sum_{e \in p} w_e + \sum_{u \in p} w_u - \sum_{b \in p} \text{penalty}(b), \tag{116}$$

where b ranges over permission or contradiction boundary crossings. The active path becomes the provenance certificate. Training penalizes missing evidence, unsupported citations, and high-scoring paths that cross permission boundaries.

I Appendix I: deployment and reporting checklist

A TropicalGT model card should include the following questions and answers. Which heads are tropical, which are softmax, and which are hybrid? Are tropical supports logged during training, validation, or both? Are support labels exact, approximate, or unavailable? What are the in-distribution and out-of-distribution length ranges? What are the value-scale ranges? What is the support recall of sparse retrieval candidate sets? What is the quantized support stability? Which structural losses are teacher-only? Which structural losses survive in the student artifact? What is the held-out bits-per-byte delta after artifact-byte accounting? What are the failure cases where tropical structure hurts? Does behavior control depend on evidence state, or only on style labels?

This checklist makes claims falsifiable. TropicalGT is useful when the structural channels explain and improve the model. If the channels are unfaithful, unstable, or too expensive, they should be removed or left in the teacher stack. The final deployment rule is deliberately conservative. A tropical module may be beautiful, interpretable, and mathematically exact on synthetic examples, yet still fail to improve held-out bits-per-byte on natural text. In that case it remains a teacher-side tool. The deployable student receives only the distilled parts that survive quantized export, support recall audits, and score-first validation.

References

[1] Baran Hashemi, Kurt Pasque, Chris Teska, and Ruriko Yoshida. Tropical Attention: Neural Algorithmic Reasoning for Combinatorial Algorithms. *arXiv:2505.17190*, 2025.

[2] Baran Hashemi, Kurt Pasque, Chris Teska, and Ruriko Yoshida. Tropical Attention: Neural Algorithmic Reasoning for Combinatorial Algorithms. NeurIPS 2025 poster, OpenReview record, 2025.

[3] Baran Hashemi et al. `Baran-phys/Tropical-Attention`: official code for Tropical Attention. GitHub repository, 2025.

[4] Baran Hashemi, Kurt Pasque, Chris Teska, and Ruriko Yoshida. Tropical Attention: Neural Algorithmic Reasoning for Combinatorial Algorithms. NeurIPS 2025 presentation slides, 2025.

[5] Ye Su and Yong Liu. Expressivity of transformers: A tropical geometry perspective. *arXiv:2604.14727*, 2026.

[6] Amelie Schreiber. Expressivity of tokenized graph transformers in terms of tropical geometry, Boolean circuits, and toric fans. Local manuscript, 2026.

[7] Diane Maclagan and Bernd Sturmfels. *Introduction to Tropical Geometry*. American Mathematical Society, 2015.

[8] Michael Joswig. *Essentials of Tropical Convexity*. American Mathematical Society, 2021.

[9] Petros Maragos, Vasileios Charisopoulos, and Emmanouil Theodosis. Tropical geometry and machine learning. *Proceedings of the IEEE*, 109(5):728–755, 2021.

[10] Liwen Zhang, Gregory Naitzat, and Lek-Heng Lim. Tropical geometry of deep neural networks. In *Proceedings of ICML*, 2018.

[11] Marie-Charlotte Brandenburg, Georg Loho, and Guido Montufar. The real tropical geometry of neural networks for binary classification. *Transactions on Machine Learning Research*, 2024.

[12] Motasem Alfarra, Adel Bibi, Hasan Hammoud, Mohamed Gaafar, and Bernard Ghanem. On the decision boundaries of neural networks: a tropical geometry perspective. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.

[13] Kurt Pasque, Christopher Teska, Ruriko Yoshida, Keiji Miura, and Jefferson Huang. Tropical decision boundaries for neural networks are robust against adversarial attacks. *arXiv:2402.00576*, 2024.

[14] Petar Velickovic, Christos Perivolaropoulos, Federico Barbero, and Razvan Pascanu. Softmax is not enough (for sharp size generalisation). *arXiv:2410.01104*, 2024/2025.

[15] Petar Velickovic, Adrià Puigdomènech Badia, David Budden, Razvan Pascanu, Andrea Banino, Misha Dashevskiy, Raia Hadsell, and Charles Blundell. The CLRS Algorithmic Reasoning Benchmark. In *Proceedings of ICML*, 2022.

[16] Andrew J. Dudzik and Petar Velickovic. Graph neural networks are dynamic programmers. In *Advances in Neural Information Processing Systems*, 2022.

[17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[18] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

[19] Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring Attention with blockwise transformers for near-infinite context. *arXiv:2310.01889*, 2023.

[20] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and Francois Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of ICML*, 2020.

[21] Marianne Akian, Stephane Gaubert, and Alexander Guterman. Tropical polyhedra are equivalent to mean payoff games. *International Journal of Algebra and Computation*, 22(1), 2012.

[22] Michael Joswig and Benjamin Schroeter. Parametric shortest-path algorithms via tropical geometry. *Mathematics of Operations Research*, 47(3):2065–2081, 2022.

[23] Stasys Jukna. *Tropical Circuit Complexity.* In related surveys and monographs on Boolean and tropical circuits, 2014.

[24] Stephane Gaubert and Ricardo Katz. The Minkowski theorem for max-plus convex sets. *Linear Algebra and its Applications*, 2011.

[25] Roan Talbut, Daniele Tramontano, Yueqi Cao, Mathias Drton, and Anthea Monod. Probability metrics for tropical spaces of different dimensions. 2024.

[26] Nicolò Nardelli et al. Neural algorithmic reasoning. Research program and overview articles, 2021–2025.

[27] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms.* MIT Press, 3rd edition, 2009.

[28] Robert W. Floyd. Algorithm 97: Shortest path. *Communications of the ACM*, 5(6):345, 1962.

[29] Stephen Warshall. A theorem on Boolean matrices. *Journal of the ACM*, 9(1):11–12, 1962.

[30] Richard Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16:87–90, 1958.

[31] Lester R. Ford. Network flow theory. RAND Corporation, 1956.

[32] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 1973.

[33] David Cox, John Little, and Hal Schenck. *Toric Varieties*. American Mathematical Society, 2011.

[34] Ezra Miller and Bernd Sturmfels. *Combinatorial Commutative Algebra*. Springer, 2005.

[35] Amelie Schreiber. Multiparameter persistent homology: generic structures and quantum computing. *arXiv:2210.11433*, 2022.

[36] David Loiseaux and Hannah Schreiber. `multipers`: multiparameter persistence for machine learning. *Journal of Open Source Software*, 9(103):6773, 2024.

[37] Gouki Minegishi, Jingyuan Feng, Hiroki Furuta, Takeshi Kojima, Yusuke Iwasawa, and Yutaka Matsuo. Emergent analogical reasoning in transformers. *arXiv:2602.01992*, 2026.

[38] Amelie Schreiber. ToricGT with Toric BGG Supervision: Graph-Token Reasoning, Hypertoric Category O, and Bits-per-byte-Constrained Training. Research manuscript, 2026.

[39] Amelie Schreiber. ToricGT II: Full-Context Dynamical, Ergodic, and Derived Persistence Geometry for Iterative Reasoning Agents. Research manuscript, 2026.

[40] Amelie Schreiber. ToricGT III: MCP Memory, Graph-Structured Vector Retrieval, and Long-Context Tropical Ring Attention. Research manuscript, 2026.

[41] Amelie Schreiber. ToricGT IV: Oracle Trajectory Classifiers for Behavior, Reasoning Quality, and Bits-per-byte Control. Research manuscript, 2026.

[42] Douglas B. West. *Introduction to Graph Theory*. Prentice Hall, 2001.

[43] Alexander Schrijver. *Combinatorial Optimization: Polyhedra and Efficiency*. Springer, 2003.

[44] R. Tyrrell Rockafellar. *Convex Analysis*. Princeton University Press, 1970.

[45] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[46] Michel Gondran and Michel Minoux. *Graphs, Dioids and Semirings: New Models and Algorithms*. Springer, 2008.

[47] Peter Butkovic. *Max-linear Systems: Theory and Algorithms*. Springer, 2010.

[48] Mehryar Mohri. Semiring frameworks and algorithms for shortest-distance problems. *Journal of Automata, Languages and Combinatorics*, 7(3):321–350, 2002.

[49] Ian Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations*, 2015.

[50] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. *International Conference on Learning Representations*, 2019.

[51] Manzil Zaheer et al. Deep Sets. In *Advances in Neural Information Processing Systems*, 2017.

[52] Chengxuan Ying et al. Do Transformers really perform badly for graph representation? *Advances in Neural Information Processing Systems*, 2021.

[53] Jinwoo Kim, Tien Dat Nguyen, Seonwoo Min, Sungjun Cho, Moontae Lee, Honglak Lee, and Seunghoon Hong. Pure Transformers are powerful graph learners. *Advances in Neural Information Processing Systems*, 2022.

[54] Will Grathwohl et al. Backpropagation through the void: Optimizing control variates for black-box gradient estimation. Related estimator methodology, 2021.

[55] Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, 2019.

[56] Charles R. Harris et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.

[57] Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[58] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. Graph of thoughts: Solving elaborate problems with large language models. In *AAAI*, 2024.

[59] Salem Lahlou, Tristan Deleu, Pablo Lemos, Dinghuai Zhang, Alexandra Volokhova, Alex Hernandez-Garcia, Lena Nehale Ezzine, Yoshua Bengio, and Nikolay Malkin. A theory of continuous generative flow networks. In *International Conference on Machine Learning*, 2023.

[60] Shengchao Liu, Chengpeng Wang, Jiarui Lu, Weili Nie, Hanchen Wang, Zhuoxinran Li, Bolei Zhou, and Jian Tang. Unsupervised discovery of steerable factors when graph deep generative models are entangled. *Transactions on Machine Learning Research*, 2024.